```
  1: ===== FILE: init_project.m =====
  2: function init_project()
  3: % ==============================
  4: % 文件名：init_project.m
  5: % 路径：S-Function_14/init_project.m
  6: % 版本号：V1.0
  7: % 最后修改时间：2025-12-10
  8: % 作者：Auto-generated
  9: % 功能描述：
 10: %   初始化 LPV-MPC 项目运行环境：
 11: %     1) 调用 project_root() 获取根目录
 12: %     2) 将 src 及其子目录递归加入 MATLAB 路径
 13: %     3) 将 simulink 目录加入路径，便于加载模型
 14: % ==============================
 15:
 16: root = project_root();
 17: addpath(root);
 18: addpath(genpath(fullfile(root, 'src')));
 19: addpath(fullfile(root, 'simulink'));
 20:
 21: fprintf('[init_project] Root: %s\n', root);
 22: end
 23:
 24: ===== FILE: project_root.m =====
 25: function root = project_root()
 26: % ==============================
 27: % 文件名：project_root.m
 28: % 路径：S-Function_14/project_root.m
 29: % 版本号：V1.0
 30: % 最后修改时间：2025-12-10
 31: % 作者：Auto-generated
 32: % 功能描述：
 33: %   返回项目根目录的绝对路径，供所有脚本构建相对路径。
 34: %   该函数应位于项目根目录，不随其他脚本迁移。
 35: % ==============================
 36:
 37: persistent cached_root
 38: if isempty(cached_root) || ~isfolder(cached_root)
 39:     [cached_root, ~] = fileparts(mfilename('fullpath'));
 40: end
 41: root = cached_root;
 42: end
 43:
 44: ===== FILE: results_dir.m =====
 45: function dir_out = results_dir(subdir)
 46: % ==============================
 47: % 文件名：results_dir.m
 48: % 路径：S-Function_14/results_dir.m
 49: % 版本号：V1.0
 50: % 最后修改时间：2025-12-10
 51: % 作者：Auto-generated
 52: % 功能描述：
 53: %   构建结果输出目录的绝对路径，并在不存在时自动创建。
 54: %   示例：out_dir = results_dir('gru/performance_offline');
 55: % ==============================
 56:
 57: if nargin < 1 || isempty(subdir)
 58:     subdir = '';
 59: end
 60:
```

```
root = project_root();
dir_out = fullfile(root, 'results', subdir);

if ~exist(dir_out, 'dir')
    mkdir(dir_out);
end
end

===== FILE: src/ModernTCN/modern_tcn_model.py =====
"""ONNX 友好的 ModernTCN-small 多任务模型。

第一版不直接照搬官方 ModernTCN 的全部组件，而是保留其核心思想：
大核 depthwise temporal convolution + pointwise channel mixing + residual。
这样导出的 ONNX 主要由 Conv1d、BatchNorm、ReLU、Linear、Mean、Concat、
Reshape/Transpose 等标准算子构成，便于后续导入 MATLAB R2024b。
"""

from __future__ import annotations

from dataclasses import dataclass, asdict
from typing import Dict, Tuple

import torch
from torch import nn


@dataclass
class ModernTCNConfig:
    """ModernTCN-small 的默认训练与结构配置。"""

    input_dim: int = 19
    seq_len: int = 128
    channels: int = 64
    blocks: int = 5
    kernel_size: int = 31
    temporal_padding: str = "same"
    dropout: float = 0.15
    expansion: int = 2
    readout_input_stats: bool = True
    turn_head_source: str = "full"
    turn_feature_indices: Tuple[int, ...] = (1, 4, 5, 6, 7, 9, 10, 11, 16)
    lambda_turn: float = 0.05
    lambda_theta: float = 0.35
    lambda_theta_flat: float = 0.20
    theta_flat_loss_mode: str = "near_zero"
    theta_flat_zero_tol_deg: float = 0.3
    lambda_theta_near_flat: float = 0.0
    theta_near_flat_deg: float = 0.5
    lambda_theta_error_excess: float = 0.0
    lambda_theta_flat_excess: float = 0.0
    lambda_theta_near_flat_excess: float = 0.0
    lambda_theta_true_zero_excess: float = 0.0
    lambda_theta_active_excess: float = 0.0
    lambda_theta_small_neg: float = 0.0
    lambda_theta_small_neg_excess: float = 0.0
    theta_excess_target_deg: float = 1.0
    theta_flat_excess_target_deg: float = 0.5
    theta_true_zero_tol_deg: float = 1e-4
    theta_small_neg_min_deg: float = -4.0
    theta_small_neg_max_deg: float = -2.0
```

```
121:     theta_gate_mode: str = "none"
122:     theta_gate_power: float = 1.0
123:     theta_gate_floor: float = 0.0
124:     main_class_multipliers: Tuple[float, float, float] = (1.20, 1.00, 0.95)
125:     turn_class_multipliers: Tuple[float, float, float] = (1.00, 1.10, 1.00)
126:     main_class_weight_method: str = "sqrt_inverse"
127:     turn_class_weight_method: str = "sqrt_inverse"
128:     main_neg_slope_weight: float = 2.0
129:     main_pos_slope_weight: float = 1.0
130:     theta_neg_weight: float = 2.0
131:     theta_pos_weight: float = 1.0
132:     turn_transition_weight: float = 1.0
133:     select_turn_weight: float = 0.30
134:     select_turn_transition_weight: float = 1.00
135:     select_turn_transition_target: float = 0.75
136:     select_turn_left_weight: float = 0.00
137:     select_turn_left_target: float = 0.80
138:     select_turn_lr_weight: float = 0.00
139:     select_turn_lr_target: float = 0.80
140:     select_theta_weight: float = 0.15
141:     select_theta_ref_deg: float = 5.0
142:     select_theta_p95_weight: float = 0.0
143:     select_theta_p95_target_deg: float = 1.0
144:     select_theta_flat_p95_weight: float = 0.0
145:     select_theta_flat_p95_target_deg: float = 1.0
146:     select_theta_near_flat_p95_weight: float = 0.0
147:     select_theta_near_flat_p95_target_deg: float = 1.0
148:     select_theta_true_zero_p95_weight: float = 0.0
149:     select_theta_true_zero_p95_target_deg: float = 1.0
150:     select_theta_flat_peak_weight: float = 0.0
151:     select_theta_flat_peak_target_deg: float = 3.0
152:     select_theta_small_neg_p95_weight: float = 0.0
153:     select_theta_small_neg_p95_target_deg: float = 1.0
154:     select_theta_extreme_p95_weight: float = 0.0
155:     select_theta_extreme_p95_target_deg: float = 1.0
156:     select_theta_edge_p95_weight: float = 0.0
157:     select_theta_edge_p95_target_deg: float = 1.2
158:     select_theta_small_nonzero_p95_weight: float = 0.0
159:     select_theta_small_nonzero_p95_target_deg: float = 1.0
160:     select_theta_flat_bias_weight: float = 0.0
161:     select_theta_flat_bias_target_deg: float = 0.2
162: 
163:     def to_dict(self) -> Dict[str, object]:
164:         return asdict(self)
165: 
166: 
167: class CausalDepthwiseConv1d(nn.Module):
168:     """Depthwise causal Conv1d with ONNX-friendly Conv + Slice semantics."""
169: 
170:     def __init__(self, channels: int, kernel_size: int) -> None:
171:         super().__init__()
172:         if kernel_size < 1:
173:             raise ValueError("kernel_size 必须 >= 1。")
174:         self.trim = kernel_size - 1
175:         self.conv = nn.Conv1d(
176:             channels,
177:             channels,
178:             kernel_size,
179:             padding=self.trim,
180:             groups=channels,
```

```
 181:         )
 182:
 183:     def forward(self, x: torch.Tensor) -> torch.Tensor:
 184:         y = self.conv(x)
 185:         if self.trim == 0:
 186:             return y
 187:         return y[..., : x.size(-1)]
 188:
 189:
 190: class ModernTCNBlock(nn.Module):
 191:     """大核 depthwise conv + pointwise MLP 的残差块。"""
 192:
 193:     def __init__(
 194:         self,
 195:         channels: int,
 196:         kernel_size: int,
 197:         dropout: float,
 198:         expansion: int,
 199:         temporal_padding: str = "same",
 200:     ) -> None:
 201:         super().__init__()
 202:         hidden = int(channels * expansion)
 203:         temporal_padding = str(temporal_padding).lower()
 204:         if temporal_padding == "same":
 205:             if kernel_size % 2 != 1:
 206:                 raise ValueError("same padding 模式下 kernel_size 必须为奇数。")
 207:             padding = kernel_size // 2
 208:             self.depthwise = nn.Conv1d(channels, channels, kernel_size, padding=padding,
groups=channels)
 209:         elif temporal_padding == "causal":
 210:             self.depthwise = CausalDepthwiseConv1d(channels, kernel_size)
 211:         else:
 212:             raise ValueError(f"未知 temporal_padding: {temporal_padding}")
 213:         self.temporal_padding = temporal_padding
 214:         self.bn1 = nn.BatchNorm1d(channels)
 215:         self.pw1 = nn.Conv1d(channels, hidden, kernel_size=1)
 216:         self.pw2 = nn.Conv1d(hidden, channels, kernel_size=1)
 217:         self.bn2 = nn.BatchNorm1d(channels)
 218:         self.act = nn.ReLU(inplace=False)
 219:         self.drop = nn.Dropout(dropout)
 220:         # layer_scale 是常数形状参数，导出 ONNX 后只对应 Mul，MATLAB 侧更容易处理。
 221:         self.layer_scale = nn.Parameter(torch.ones(1, channels, 1) * 1e-2)
 222:
 223:     def forward(self, x: torch.Tensor) -> torch.Tensor:
 224:         residual = x
 225:         y = self.depthwise(x)
 226:         y = self.bn1(y)
 227:         y = self.act(y)
 228:         y = self.pw1(y)
 229:         y = self.act(y)
 230:         y = self.drop(y)
 231:         y = self.pw2(y)
 232:         y = self.bn2(y)
 233:         y = self.drop(y)
 234:         return residual + y * self.layer_scale
 235:
 236:
 237: class ModernTCNSmall(nn.Module):
 238:     """固定输入 `[batch, time=128, features=19]` 的三输出多任务网络。"""
 239:
 240:     def __init__(self, cfg: ModernTCNConfig) -> None:
```

```
241:         super().__init__()
242:         self.cfg = cfg
243:         self.stem = nn.Sequential(
244:             nn.Conv1d(cfg.input_dim, cfg.channels, kernel_size=1),
245:             nn.BatchNorm1d(cfg.channels),
246:             nn.ReLU(inplace=False),
247:         )
248:         self.blocks = nn.ModuleList(
249:             [
250:                 ModernTCNBlock(
251:                     cfg.channels,
252:                     cfg.kernel_size,
253:                     cfg.dropout,
254:                     cfg.expansion,
255:                     temporal_padding=cfg.temporal_padding,
256:                 )
257:                 for _ in range(cfg.blocks)
258:             ]
259:         )
260:         feature_dim = cfg.channels * 3
261:         if cfg.readout_input_stats:
262:             feature_dim += cfg.input_dim * 5
263:         input_stats_dim = cfg.input_dim * 5
264:         turn_source = cfg.turn_head_source.lower()
265:         if turn_source == "full":
266:             turn_dim = feature_dim
267:         elif turn_source == "inputstats":
268:             turn_dim = input_stats_dim
269:         elif turn_source == "kinematic_stats":
270:             turn_indices = self._make_turn_stats_indices(cfg.input_dim, cfg.turn_feature_indices)
271:             self.register_buffer("turn_stats_indices", turn_indices, persistent=False)
272:             turn_dim = int(turn_indices.numel())
273:         else:
274:             raise ValueError(f"未知 turn_head_source: {cfg.turn_head_source}")
275:         self.turn_head_source = turn_source
276:
277:         # 三个任务头严格对应 MATLAB 端后处理契约。
278:         self.main_head = nn.Linear(feature_dim, 3)
279:         self.turn_head = nn.Sequential(
280:             nn.Linear(turn_dim, 64),
281:             nn.ReLU(inplace=False),
282:             nn.Linear(64, 3),
283:         )
284:         self.theta_head = nn.Linear(feature_dim, 1)
285:
286:     def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
287:         # x: [B, T, F]；Conv1d 使用 [B, F, T]。
288:         x_ct = x.transpose(1, 2)
289:         z = self.stem(x_ct)
290:         for block in self.blocks:
291:             z = block(z)
292:
293:         h_last = z[:, :, -1]
294:         h_mean = z.mean(dim=2)
295:         h_max = z.amax(dim=2)
296:         pieces = [h_last, h_mean, h_max]
297:         input_stats = self._input_stats(x_ct)
298:         if self.cfg.readout_input_stats:
299:             pieces.append(input_stats)
300:         h = torch.cat(pieces, dim=1)
```

```
301:         logits_main = self.main_head(h)
302:         if self.turn_head_source == "full":
303:             h_turn = h
304:         elif self.turn_head_source == "inputstats":
305:             h_turn = input_stats
306:         else:
307:             h_turn = input_stats.index_select(1, self.turn_stats_indices)
308:         logits_turn = self.turn_head(h_turn)
309:         theta_hat = self.theta_head(h)
310:         theta_hat = self._apply_theta_gate(theta_hat, logits_main)
311:         return logits_main, logits_turn, theta_hat
312:
313:     def _apply_theta_gate(self, theta_hat: torch.Tensor, logits_main: torch.Tensor) ->
torch.Tensor:
314:         """Optionally suppress theta when the main head is confident the window is flat/stall."""
315:
316:         mode = str(getattr(self.cfg, "theta_gate_mode", "none")).lower()
317:         if mode in ("", "none"):
318:             return theta_hat
319:         if mode != "main_slope_prob":
320:             raise ValueError(f"未知 theta_gate_mode: {self.cfg.theta_gate_mode}")
321:         slope_prob = torch.softmax(logits_main, dim=1)[:, 2:3]
322:         floor = float(getattr(self.cfg, "theta_gate_floor", 0.0))
323:         power = float(getattr(self.cfg, "theta_gate_power", 1.0))
324:         gate = floor + (1.0 - floor) * torch.pow(torch.clamp(slope_prob, min=1e-6), power)
325:         return theta_hat * gate
326:
327:     @staticmethod
328:     def _input_stats(x_ct: torch.Tensor) -> torch.Tensor:
329:         """复用窗口级统计线索，但仍只来自同一 19 维输入窗口。"""
330:
331:         x_last = x_ct[:, :, -1]
332:         x_mean = x_ct.mean(dim=2)
333:         x_std = torch.sqrt(torch.mean((x_ct - x_mean.unsqueeze(2)) ** 2, dim=2) + 1e-8)
334:         x_max = x_ct.amax(dim=2)
335:         x_min = x_ct.amin(dim=2)
336:         return torch.cat([x_last, x_mean, x_std, x_max, x_min], dim=1)
337:
338:     @staticmethod
339:     def _make_turn_stats_indices(input_dim: int, feature_indices: Tuple[int, ...]) -> torch.Tensor:
340:         """Return gather indices for [last, mean, std, max, min] input stats."""
341:
342:         idx = []
343:         for stat_block in range(5):
344:             base = stat_block * input_dim
345:             idx.extend(base + int(i) for i in feature_indices)
346:         return torch.tensor(idx, dtype=torch.long)
347:
348:
349: def build_model_from_checkpoint_dict(ckpt: Dict[str, object]) -> ModernTCNSmall:
350:     """按 checkpoint 中的配置恢复模型结构。"""
351:
352:     cfg_dict = dict(ckpt["model_config"])
353:     cfg_dict.setdefault("temporal_padding", "same")
354:     cfg_dict["main_class_multipliers"] = tuple(cfg_dict["main_class_multipliers"])
355:     cfg_dict["turn_class_multipliers"] = tuple(cfg_dict["turn_class_multipliers"])
356:     if "turn_feature_indices" in cfg_dict:
357:         cfg_dict["turn_feature_indices"] = tuple(cfg_dict["turn_feature_indices"])
358:     cfg = ModernTCNConfig(**cfg_dict)
359:     model = ModernTCNSmall(cfg)
360:     model.load_state_dict(ckpt["model_state"])
```

```
    return model


===== FILE: src/ModernTCN/modern_tcn_data.py =====
"""读取固定 ModernTCN 数据集，并保持 MATLAB 侧已有 split/scaler 不变。

本文件默认读取 `data/tcn/ModernTCN_dataset_v4_industrial.mat` 中已经生成好的
窗口、标签、样本权重和元信息。这里不能重新划分 train/val/test，也不能
重新拟合 scaler；Python 端训练必须直接使用 MAT 文件里的归一化后 X。
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional

import h5py
import numpy as np
import torch
from torch.utils.data import Dataset


DATASET_RELATIVE_PATH = Path("data") / "tcn" / "ModernTCN_dataset_v4_industrial.mat"


@dataclass(frozen=True)
class ModernTCNContract:
    """记录第一阶段实验必须锁定的数据契约。"""

    dataset_file: str
    seq_len: int
    input_dim: int
    output_contract: str
    split_policy: str
    scaler_policy: str
    vehicle_type: str = "diagonal_dual_steer_drive_agv"
    active_drive_steer_wheels: str = "LF,RR"
    passive_support_wheels: str = "RF,LR"
    feature_policy: str = "keep_current_algorithm_inputs_unchanged"
    label_time_policy: str = "current_window_end"
    horizon_steps: int = 0
    horizon_seconds: float = 0.0
    confidence_policy: str = "derive_classification_confidence_from_softmax_and_export"


@dataclass
class SplitArrays:
    """单个 split 的全部训练/评估数组。"""

    X: np.ndarray
    y_main: np.ndarray
    y_turn: np.ndarray
    y_theta: np.ndarray
    mask_theta: np.ndarray
    main_weight: np.ndarray
    turn_weight: np.ndarray
    theta_weight: np.ndarray
    turn_purity: np.ndarray
    turn_transition: np.ndarray
    run_id: np.ndarray
```

```
421:
422:
423: class AGVWindowDataset(Dataset):
424:     """PyTorch Dataset，直接封装 MAT 文件中的窗口数据。"""
425:
426:     def __init__(self, split: SplitArrays) -> None:
427:         self.split = split
428:
429:     def __len__(self) -> int:
430:         return int(self.split.X.shape[0])
431:
432:     def __getitem__(self, idx: int) -> Dict[str, torch.Tensor]:
433:         # 这里保持输入格式为 [time, feature]，模型内部再转成 Conv1d 需要的 [B, C, T]。
434:         return {
435:             "X": torch.from_numpy(self.split.X[idx]).float(),
436:             "y_main": torch.tensor(self.split.y_main[idx], dtype=torch.long),
437:             "y_turn": torch.tensor(self.split.y_turn[idx], dtype=torch.long),
438:             "y_theta": torch.tensor(self.split.y_theta[idx], dtype=torch.float32),
439:             "mask_theta": torch.tensor(self.split.mask_theta[idx], dtype=torch.float32),
440:             "main_weight": torch.tensor(self.split.main_weight[idx], dtype=torch.float32),
441:             "turn_weight": torch.tensor(self.split.turn_weight[idx], dtype=torch.float32),
442:             "theta_weight": torch.tensor(self.split.theta_weight[idx], dtype=torch.float32),
443:             "turn_purity": torch.tensor(self.split.turn_purity[idx], dtype=torch.float32),
444:             "turn_transition": torch.tensor(self.split.turn_transition[idx], dtype=torch.bool),
445:             "run_id": torch.tensor(self.split.run_id[idx], dtype=torch.float32),
446:         }
447:
448:
449: def find_project_root(start: Optional[Path] = None) -> Path:
450:     """从当前文件向上寻找项目根目录。"""
451:
452:     if start is None:
453:         start = Path(__file__).resolve()
454:     start = start.resolve()
455:     candidates: Iterable[Path] = (start, *start.parents)
456:     for p in candidates:
457:         if (p / "init_project.m").exists() and (p / "data" / "tcn").exists():
458:             return p
459:     raise FileNotFoundError("无法定位项目根目录：未找到 init_project.m 和 data/tcn。")
460:
461:
462: def default_dataset_file(project_root: Optional[Path] = None) -> Path:
463:     """返回当前默认 ModernTCN 数据集路径。"""
464:
465:     root = project_root or find_project_root()
466:     return root / DATASET_RELATIVE_PATH
467:
468:
469: def load_modern_tcn_dataset(
470:     dataset_file: Optional[Path] = None,
471:     limit_train: int = 0,
472:     limit_val: int = 0,
473:     limit_test: int = 0,
474: ) -> Dict[str, object]:
475:     """读取固定 ModernTCN 数据集。
476:
477:     MATLAB v7.3 MAT 文件本质上是 HDF5；数值数组在 HDF5 中维度顺序和 MATLAB
478:     `size(dataset.X_train) == [N, 128, 19]` 相反，因此这里显式转回
479:     `[window, time, feature]`。
480:     """
```

```
 481:
 482:     dataset_file = Path(dataset_file) if dataset_file else default_dataset_file()
 483:     if not dataset_file.exists():
 484:         raise FileNotFoundError(f"找不到固定 ModernTCN 数据集：{dataset_file}")
 485:
 486:     with h5py.File(dataset_file, "r") as f:
 487:         root = f["dataset"]
 488:         train = _read_split(root, "train", limit_train)
 489:         val = _read_split(root, "val", limit_val)
 490:         test = _read_split(root, "test", limit_test)
 491:         feat_names = _read_feat_names(f, root)
 492:         scaler = _read_scaler(root)
 493:
 494:     contract = ModernTCNContract(
 495:         dataset_file=str(dataset_file),
 496:         seq_len=int(train.X.shape[1]),
 497:         input_dim=int(train.X.shape[2]),
 498:         output_contract="logits_main3_logits_turn3_theta1",
 499:         split_policy="use_existing_run_level_split_from_mat",
 500:         scaler_policy="use_existing_normalized_X_and_existing_scaler_only",
 501:     )
 502:
 503:     _check_contract(contract)
 504:     return {
 505:         "train": train,
 506:         "val": val,
 507:         "test": test,
 508:         "feat_names": feat_names,
 509:         "scaler": scaler,
 510:         "contract": contract,
 511:     }
 512:
 513:
 514: def class_weights(labels: np.ndarray, num_classes: int, method: str, multipliers: List[float]) ->
torch.Tensor:
 515:     """复刻 MATLAB baseline 中的类别权重计算方式。"""
 516:
 517:     labels = np.asarray(labels).reshape(-1)
 518:     counts = np.array([(labels == i).sum() for i in range(num_classes)], dtype=np.float64)
 519:     counts_safe = np.maximum(counts, 1.0)
 520:     method = method.lower()
 521:     if method == "inverse":
 522:         w = 1.0 / counts_safe
 523:     elif method == "sqrt_inverse":
 524:         w = 1.0 / np.sqrt(counts_safe)
 525:     elif method == "balanced":
 526:         w = counts.sum() / (num_classes * counts_safe)
 527:     else:
 528:         w = np.ones(num_classes, dtype=np.float64)
 529:     w[counts == 0] = 0.0
 530:     if np.any(w > 0):
 531:         w = w / np.mean(w[w > 0])
 532:     else:
 533:         w = np.ones(num_classes, dtype=np.float64)
 534:     w = w * np.asarray(multipliers, dtype=np.float64)
 535:     if np.any(w > 0):
 536:         w = w / np.mean(w[w > 0])
 537:     return torch.tensor(w, dtype=torch.float32)
 538:
 539:
 540: def _read_split(root: h5py.Group, split_name: str, limit: int) -> SplitArrays:
```

```
    X = _read_h5_array(root[f"X_{split_name}"])
    # MATLAB 主工况标签为 1/2/3；PyTorch CE 使用 0/1/2。
    y_main = _read_vector(root[f"y_main_{split_name}"]).astype(np.int64) - 1
    # MATLAB 转弯标签为 -1/0/1；PyTorch CE 使用 0/1/2。
    y_turn = _read_vector(root[f"y_turn_{split_name}"]).astype(np.int64) + 1
    y_theta = _read_vector(root[f"y_theta_{split_name}"]).astype(np.float32)
    mask_theta = _read_vector(root[f"mask_theta_{split_name}"]).astype(np.float32)
    main_weight = _read_vector(root[f"main_sample_weight_{split_name}"]).astype(np.float32)
    turn_weight = _read_vector(root[f"turn_sample_weight_{split_name}"]).astype(np.float32)
    theta_weight = _read_vector(root[f"theta_sample_weight_{split_name}"]).astype(np.float32)
    turn_purity = _read_vector(root[f"turn_purity_{split_name}"]).astype(np.float32)
    turn_transition = _read_vector(root[f"turn_transition_{split_name}"]).astype(bool)
    run_id = _read_vector(root[f"run_id_{split_name}"]).astype(np.float32)

    if limit and limit > 0:
        sl = slice(0, int(limit))
        X = X[sl]
        y_main = y_main[sl]
        y_turn = y_turn[sl]
        y_theta = y_theta[sl]
        mask_theta = mask_theta[sl]
        main_weight = main_weight[sl]
        turn_weight = turn_weight[sl]
        theta_weight = theta_weight[sl]
        turn_purity = turn_purity[sl]
        turn_transition = turn_transition[sl]
        run_id = run_id[sl]

    return SplitArrays(
        X=X,
        y_main=y_main,
        y_turn=y_turn,
        y_theta=y_theta,
        mask_theta=mask_theta,
        main_weight=main_weight,
        turn_weight=turn_weight,
        theta_weight=theta_weight,
        turn_purity=turn_purity,
        turn_transition=turn_transition,
        run_id=run_id,
    )


def _read_h5_array(ds: h5py.Dataset) -> np.ndarray:
    raw = np.asarray(ds, dtype=np.float32)
    if raw.ndim != 3:
        raise ValueError(f"期望 3D 窗口数组，实际 shape={raw.shape}")
    return np.transpose(raw, (2, 1, 0)).copy()


def _read_vector(ds: h5py.Dataset) -> np.ndarray:
    return np.asarray(ds).reshape(-1).copy()


def _read_scaler(root: h5py.Group) -> Dict[str, np.ndarray]:
    scaler_group = root["scaler"]
    return {
        "mean": _read_vector(scaler_group["mean"]).astype(np.float32),
        "std": _read_vector(scaler_group["std"]).astype(np.float32),
    }
```

```
 601:
 602:
 603: def _read_feat_names(f: h5py.File, root: h5py.Group) -> List[str]:
 604:     names: List[str] = []
 605:     refs = np.asarray(root["feat_names"]).reshape(-1)
 606:     for ref in refs:
 607:         names.append(_decode_matlab_char(f[ref]))
 608:     return names
 609:
 610:
 611: def _decode_matlab_char(ds: h5py.Dataset) -> str:
 612:     raw = np.asarray(ds).reshape(-1)
 613:     return "".join(chr(int(c)) for c in raw if int(c) != 0)
 614:
 615:
 616: def _check_contract(contract: ModernTCNContract) -> None:
 617:     if contract.seq_len != 128:
 618:         raise ValueError(f"ModernTCN 第一阶段要求 seq_len=128，实际为 {contract.seq_len}")
 619:     if contract.input_dim != 19:
 620:         raise ValueError(f"ModernTCN 第一阶段要求 input_dim=19，实际为 {contract.input_dim}")
 621:     if contract.horizon_steps != 0:
 622:         raise ValueError(f"ModernTCN 当前模型固定为当前状态估计 horizon_steps=0，实际为
{contract.horizon_steps}")
 623:
 624: ===== FILE: src/ModernTCN/modern_tcn_metrics.py =====
 625: """ModernTCN 第一阶段指标、损失和门槛判定。
 626:
 627: 指标定义和 `run_TCN_GRU_transition_rich_v3_baseline.m` 对齐：输出
 628: acc_main、acc_turn、acc_turn_transition、theta_mae_deg、flat/stall/slope
 629: recall、uphill/downhill recall。seed42 门槛只用于决定是否进入三 seed，
 630: 不用于训练集或验证集重划分。
 631: """
 632:
 633: from __future__ import annotations
 634:
 635: from dataclasses import dataclass
 636: from typing import Dict, Iterable, List, Tuple
 637:
 638: import numpy as np
 639: import torch
 640: import torch.nn.functional as F
 641:
 642:
 643: @dataclass(frozen=True)
 644: class GateThresholds:
 645:     acc_main: float = 0.90
 646:     flat_recall: float = 0.90
 647:     slope_recall: float = 0.88
 648:     acc_turn_transition: float = 0.75
 649:     theta_mae_deg: float = 0.70
 650:
 651:
 652: def multitask_loss(
 653:     logits_main: torch.Tensor,
 654:     logits_turn: torch.Tensor,
 655:     theta_hat: torch.Tensor,
 656:     batch: Dict[str, torch.Tensor],
 657:     class_weights_main: torch.Tensor,
 658:     class_weights_turn: torch.Tensor,
 659:     cfg,
 660: ) -> Tuple[torch.Tensor, Dict[str, float]]:
```

```
661:     """计算与 MATLAB TCN/GRU baseline 对齐的多任务损失。"""
662:
663:     y_main = batch["y_main"]
664:     y_turn = batch["y_turn"]
665:     theta = batch["y_theta"]
666:     mask_theta = batch["mask_theta"]
667:
668:     main_sample_weight = batch["main_weight"] * _main_slope_sample_weight(y_main, theta, cfg)
669:     turn_sample_weight = batch["turn_weight"] * (
670:         1.0 + (cfg.turn_transition_weight - 1.0) * batch["turn_transition"].float()
671:     )
672:     theta_sample_weight = batch["theta_weight"] * _theta_sign_weight(theta, cfg)
673:
674:     loss_main = _weighted_ce(logits_main, y_main, class_weights_main, main_sample_weight)
675:     loss_turn = _weighted_ce(logits_turn, y_turn, class_weights_turn, turn_sample_weight)
676:     theta_hat = theta_hat.reshape(-1)
677:     loss_theta = _masked_weighted_mse(theta_hat, theta, mask_theta, theta_sample_weight)
678:     near_flat_limit = torch.deg2rad(torch.tensor(float(getattr(cfg, "theta_near_flat_deg", 0.5)),
device=theta.device))
679:     near_flat_mask = (torch.abs(theta) <= near_flat_limit).float() * mask_theta
680:     loss_theta_near_flat = _masked_weighted_mse(
681:         theta_hat, torch.zeros_like(theta), near_flat_mask, theta_sample_weight
682:     )
683:     true_zero_limit = torch.deg2rad(
684:         torch.tensor(float(getattr(cfg, "theta_true_zero_tol_deg", 1e-4)), device=theta.device)
685:     )
686:     true_zero_mask = (torch.abs(theta) <= true_zero_limit).float() * mask_theta
687:     flat_zero_mask = _theta_flat_zero_mask(theta, y_main, mask_theta, near_flat_mask,
true_zero_mask, cfg)
688:     loss_theta_flat = _masked_weighted_mse(theta_hat, torch.zeros_like(theta), flat_zero_mask,
theta_sample_weight)
689:     active_mask = (torch.abs(theta) >= torch.deg2rad(torch.tensor(2.0,
device=theta.device))).float() * mask_theta
690:     small_neg_min = torch.deg2rad(
691:         torch.tensor(float(getattr(cfg, "theta_small_neg_min_deg", -4.0)), device=theta.device)
692:     )
693:     small_neg_max = torch.deg2rad(
694:         torch.tensor(float(getattr(cfg, "theta_small_neg_max_deg", -2.0)), device=theta.device)
695:     )
696:     small_neg_mask = ((theta >= small_neg_min) & (theta <= small_neg_max)).float() * mask_theta
697:     loss_theta_error_excess = _masked_weighted_abs_excess_mse(
698:         theta_hat, theta, mask_theta, theta_sample_weight, float(getattr(cfg,
"theta_excess_target_deg", 1.0))
699:     )
700:     loss_theta_flat_excess = _masked_weighted_abs_excess_mse(
701:         theta_hat,
702:         torch.zeros_like(theta),
703:         flat_zero_mask,
704:         theta_sample_weight,
705:         float(getattr(cfg, "theta_flat_excess_target_deg", 0.5)),
706:     )
707:     loss_theta_near_flat_excess = _masked_weighted_abs_excess_mse(
708:         theta_hat,
709:         torch.zeros_like(theta),
710:         near_flat_mask,
711:         theta_sample_weight,
712:         float(getattr(cfg, "theta_flat_excess_target_deg", 0.5)),
713:     )
714:     loss_theta_true_zero_excess = _masked_weighted_abs_excess_mse(
715:         theta_hat,
716:         torch.zeros_like(theta),
717:         true_zero_mask,
718:         theta_sample_weight,
719:         float(getattr(cfg, "theta_flat_excess_target_deg", 0.5)),
720:     )
```

```
 721:     loss_theta_active_excess = _masked_weighted_abs_excess_mse(
 722:         theta_hat, theta, active_mask, theta_sample_weight, float(getattr(cfg,
"theta_excess_target_deg", 1.0))
 723:     )
 724:     loss_theta_small_neg = _masked_weighted_mse(theta_hat, theta, small_neg_mask,
theta_sample_weight)
 725:     loss_theta_small_neg_excess = _masked_weighted_abs_excess_mse(
 726:         theta_hat, theta, small_neg_mask, theta_sample_weight, float(getattr(cfg,
"theta_excess_target_deg", 1.0))
 727:     )
 728:
 729:     total = (
 730:         loss_main
 731:         + cfg.lambda_turn * loss_turn
 732:         + cfg.lambda_theta * loss_theta
 733:         + cfg.lambda_theta_flat * loss_theta_flat
 734:         + float(getattr(cfg, "lambda_theta_near_flat", 0.0)) * loss_theta_near_flat
 735:         + float(getattr(cfg, "lambda_theta_error_excess", 0.0)) * loss_theta_error_excess
 736:         + float(getattr(cfg, "lambda_theta_flat_excess", 0.0)) * loss_theta_flat_excess
 737:         + float(getattr(cfg, "lambda_theta_near_flat_excess", 0.0)) * loss_theta_near_flat_excess
 738:         + float(getattr(cfg, "lambda_theta_true_zero_excess", 0.0)) * loss_theta_true_zero_excess
 739:         + float(getattr(cfg, "lambda_theta_active_excess", 0.0)) * loss_theta_active_excess
 740:         + float(getattr(cfg, "lambda_theta_small_neg", 0.0)) * loss_theta_small_neg
 741:         + float(getattr(cfg, "lambda_theta_small_neg_excess", 0.0)) * loss_theta_small_neg_excess
 742:     )
 743:     parts = {
 744:         "loss_total": float(total.detach().cpu()),
 745:         "loss_main": float(loss_main.detach().cpu()),
 746:         "loss_turn": float(loss_turn.detach().cpu()),
 747:         "loss_theta": float(loss_theta.detach().cpu()),
 748:         "loss_theta_flat": float(loss_theta_flat.detach().cpu()),
 749:         "loss_theta_near_flat": float(loss_theta_near_flat.detach().cpu()),
 750:         "loss_theta_error_excess": float(loss_theta_error_excess.detach().cpu()),
 751:         "loss_theta_flat_excess": float(loss_theta_flat_excess.detach().cpu()),
 752:         "loss_theta_near_flat_excess": float(loss_theta_near_flat_excess.detach().cpu()),
 753:         "loss_theta_true_zero_excess": float(loss_theta_true_zero_excess.detach().cpu()),
 754:         "loss_theta_active_excess": float(loss_theta_active_excess.detach().cpu()),
 755:         "loss_theta_small_neg": float(loss_theta_small_neg.detach().cpu()),
 756:         "loss_theta_small_neg_excess": float(loss_theta_small_neg_excess.detach().cpu()),
 757:     }
 758:     return total, parts
 759:
 760:
 761: def _theta_flat_zero_mask(
 762:     theta: torch.Tensor,
 763:     y_main: torch.Tensor,
 764:     mask_theta: torch.Tensor,
 765:     near_flat_mask: torch.Tensor,
 766:     true_zero_mask: torch.Tensor,
 767:     cfg,
 768: ) -> torch.Tensor:
 769:     """Return the samples where the auxiliary flat-theta loss may force zero.
 770:
 771:     ``main_flat`` is kept for legacy reproduction. New balanced training uses
 772:     ``near_zero`` so |theta|<2 deg can still be classified as flat without
 773:     teaching the regression head to collapse those angles to zero.
 774:     """
 775:
 776:     mode = str(getattr(cfg, "theta_flat_loss_mode", "near_zero")).lower()
 777:     if mode in ("", "none", "off"):
 778:         return torch.zeros_like(mask_theta)
 779:     if mode in ("main_flat", "flat", "legacy"):
 780:         return (y_main == 0).float() * mask_theta
```

```
    if mode in ("near_flat", "theta_near_flat"):
        return near_flat_mask
    if mode in ("true_zero", "zero"):
        return true_zero_mask
    if mode in ("near_zero", "very_near_zero"):
        tol = torch.deg2rad(
            torch.tensor(float(getattr(cfg, "theta_flat_zero_tol_deg", 0.3)), device=theta.device)
        )
        return (torch.abs(theta) <= tol).float() * mask_theta
    raise ValueError(f"Unknown theta_flat_loss_mode: {mode}")


def compute_metrics(
    logits_main: np.ndarray,
    logits_turn: np.ndarray,
    theta_hat: np.ndarray,
    split,
    loss_total: float = float("nan"),
) -> Dict[str, object]:
    """根据完整 split 的预测结果计算评估指标。"""

    prob_main = _softmax_np(np.asarray(logits_main))
    prob_turn = _softmax_np(np.asarray(logits_turn))
    pred_main = prob_main.argmax(axis=1)
    pred_turn_cls = prob_turn.argmax(axis=1)
    pred_turn_raw = pred_turn_cls - 1
    y_main = split.y_main.reshape(-1)
    y_turn_raw = split.y_turn.reshape(-1) - 1
    theta_true = split.y_theta.reshape(-1)
    theta_hat = np.asarray(theta_hat).reshape(-1)
    main_confidence = prob_main.max(axis=1)
    turn_confidence = prob_turn.max(axis=1)

    cm_main = _confusion_matrix(y_main, pred_main, 3)
    cm_turn = _confusion_matrix(y_turn_raw + 1, pred_turn_raw + 1, 3)
    recall_main = np.diag(cm_main) / np.maximum(cm_main.sum(axis=1), 1)
    recall_turn = np.diag(cm_turn) / np.maximum(cm_turn.sum(axis=1), 1)
    transition_mask = split.turn_transition.astype(bool)
    pure_mask = np.isfinite(split.turn_purity) & (split.turn_purity >= 0.8) & (~transition_mask)
    slope_idx = np.where(split.mask_theta.reshape(-1) == 1)[0]

    metrics: Dict[str, object] = {
        "loss_total": float(loss_total),
        "acc_main": float(np.mean(pred_main == y_main)),
        "acc_turn": float(np.mean(pred_turn_raw == y_turn_raw)),
        "acc_turn_pure": _masked_acc(pred_turn_raw, y_turn_raw, pure_mask),
        "acc_turn_transition": _masked_acc(pred_turn_raw, y_turn_raw, transition_mask),
        "flat_recall": float(recall_main[0]),
        "stall_recall": float(recall_main[1]),
        "slope_recall": float(recall_main[2]),
        "recall_main": [float(x) for x in recall_main],
        "turn_right_recall": float(recall_turn[0]),
        "turn_straight_recall": float(recall_turn[1]),
        "turn_left_recall": float(recall_turn[2]),
        "recall_turn": [float(x) for x in recall_turn],
        "cm_turn": cm_turn.tolist(),
        "n_turn_transition": int(transition_mask.sum()),
        "n_turn_pure": int(pure_mask.sum()),
        "cm_main": cm_main.tolist(),
    }
```

```
 841:     metrics.update(_confidence_summary("main", main_confidence, pred_main == y_main))
 842:     metrics.update(_confidence_summary("turn", turn_confidence, pred_turn_raw == y_turn_raw))
 843:
 844:     if slope_idx.size == 0:
 845:         metrics.update(
 846:             {
 847:                 "theta_mae_rad": 0.0,
 848:                 "theta_mae_deg": 0.0,
 849:                 "uphill_recall": float("nan"),
 850:                 "downhill_recall": float("nan"),
 851:                 "slope_sign_acc": float("nan"),
 852:             }
 853:         )
 854:     else:
 855:         theta_err = np.abs(theta_hat[slope_idx] - theta_true[slope_idx])
 856:         uphill_idx = slope_idx[theta_true[slope_idx] > 0]
 857:         downhill_idx = slope_idx[theta_true[slope_idx] < 0]
 858:         metrics.update(
 859:             {
 860:                 "theta_mae_rad": float(theta_err.mean()),
 861:                 "theta_mae_deg": float(np.rad2deg(theta_err.mean())),
 862:                 "uphill_recall": _slope_sub_recall(pred_main, uphill_idx),
 863:                 "downhill_recall": _slope_sub_recall(pred_main, downhill_idx),
 864:                 "slope_sign_acc": float(np.mean(np.sign(theta_hat[slope_idx]) ==
np.sign(theta_true[slope_idx]))),
 865:             }
 866:         )
 867:     metrics.update(_theta_control_metrics(theta_hat, theta_true, y_main, y_turn_raw,
split.mask_theta.reshape(-1)))
 868:     return metrics
 869:
 870:
 871: def selection_score(metrics: Dict[str, object], cfg=None) -> float:
 872:     """验证集选模分数：主工况边界优先，兼顾转弯过渡和 theta。"""
 873:
 874:     select_theta_weight = float(getattr(cfg, "select_theta_weight", 0.15))
 875:     select_theta_ref_deg = max(float(getattr(cfg, "select_theta_ref_deg", 5.0)), 1e-6)
 876:     theta_norm = float(metrics["theta_mae_deg"]) / select_theta_ref_deg
 877:     flat_penalty = max(0.0, 0.90 - float(metrics["flat_recall"]))
 878:     slope_penalty = max(0.0, 0.88 - float(metrics["slope_recall"]))
 879:     turn_t = float(metrics["acc_turn_transition"])
 880:     select_turn_weight = float(getattr(cfg, "select_turn_weight", 0.30))
 881:     select_turn_t_weight = float(getattr(cfg, "select_turn_transition_weight", 1.00))
 882:     select_turn_t_target = float(getattr(cfg, "select_turn_transition_target", 0.75))
 883:     select_turn_left_weight = float(getattr(cfg, "select_turn_left_weight", 0.00))
 884:     select_turn_left_target = float(getattr(cfg, "select_turn_left_target", 0.80))
 885:     select_turn_lr_weight = float(getattr(cfg, "select_turn_lr_weight", 0.00))
 886:     select_turn_lr_target = float(getattr(cfg, "select_turn_lr_target", 0.80))
 887:     turn_t_penalty = 0.0 if np.isnan(turn_t) else max(0.0, select_turn_t_target - turn_t)
 888:     turn_right = float(metrics.get("turn_right_recall", float("nan")))
 889:     turn_left = float(metrics.get("turn_left_recall", float("nan")))
 890:     turn_left_penalty = 0.0 if np.isnan(turn_left) else max(0.0, select_turn_left_target -
turn_left)
 891:     if np.isnan(turn_right) or np.isnan(turn_left):
 892:         turn_lr_penalty = 0.0
 893:     else:
 894:         turn_lr_penalty = max(0.0, select_turn_lr_target - min(turn_right, turn_left))
 895:     flat_p95 = float(metrics.get("theta_flat_abs_p95_deg", float("nan")))
 896:     flat_p95_target = float(getattr(cfg, "select_theta_flat_p95_target_deg", 1.0))
 897:     flat_p95_penalty = 0.0 if np.isnan(flat_p95) else max(0.0, flat_p95 - flat_p95_target) / 5.0
 898:     theta_p95_penalty = _metric_excess_penalty(
 899:         metrics, "theta_abs_le_8_p95_abs_err_deg", float(getattr(cfg,
"select_theta_p95_target_deg", 1.0)), 5.0
 900:     )
```

```
    near_flat_p95_penalty = _metric_excess_penalty(
        metrics,
        "theta_near_flat_abs_p95_deg",
        float(getattr(cfg, "select_theta_near_flat_p95_target_deg", 1.0)),
        5.0,
    )
    true_zero_p95_penalty = _metric_excess_penalty(
        metrics,
        "theta_true_zero_abs_p95_deg",
        float(getattr(cfg, "select_theta_true_zero_p95_target_deg", 1.0)),
        5.0,
    )
    flat_peak_penalty = _metric_excess_penalty(
        metrics,
        "theta_flat_abs_max_deg",
        float(getattr(cfg, "select_theta_flat_peak_target_deg", 3.0)),
        10.0,
    )
    small_neg_p95_penalty = _metric_excess_penalty(
        metrics,
        "theta_neg_4_2_p95_abs_err_deg",
        float(getattr(cfg, "select_theta_small_neg_p95_target_deg", 1.0)),
        5.0,
    )
    extreme_p95_penalty = 0.5 * (
        _metric_excess_penalty(
            metrics,
            "theta_neg_8_6_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_extreme_p95_target_deg", 1.0)),
            5.0,
        )
        + _metric_excess_penalty(
            metrics,
            "theta_pos_6_8_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_extreme_p95_target_deg", 1.0)),
            5.0,
        )
    )
    edge_p95_penalty = 0.5 * (
        _metric_excess_penalty(
            metrics,
            "theta_neg_10_8_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_edge_p95_target_deg", 1.2)),
            5.0,
        )
        + _metric_excess_penalty(
            metrics,
            "theta_pos_8_10_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_edge_p95_target_deg", 1.2)),
            5.0,
        )
    )
    small_nonzero_p95_penalty = 0.5 * (
        _metric_excess_penalty(
            metrics,
            "theta_neg_2_0p5_p95_abs_err_deg",
            float(getattr(cfg, "select_theta_small_nonzero_p95_target_deg", 1.0)),
            5.0,
        )
        + _metric_excess_penalty(
```

```
 961:             metrics,
 962:             "theta_pos_0p5_2_p95_abs_err_deg",
 963:             float(getattr(cfg, "select_theta_small_nonzero_p95_target_deg", 1.0)),
 964:             5.0,
 965:         )
 966:     )
 967:     flat_bias = abs(float(metrics.get("theta_flat_bias_deg", float("nan"))))
 968:     flat_bias_target = float(getattr(cfg, "select_theta_flat_bias_target_deg", 0.2))
 969:     flat_bias_penalty = 0.0 if np.isnan(flat_bias) else max(0.0, flat_bias - flat_bias_target) /
5.0
 970:     return (
 971:         float(metrics["loss_total"])
 972:         + 1.00 * (1.0 - float(metrics["acc_main"]))
 973:         + select_turn_weight * (1.0 - float(metrics["acc_turn"]))
 974:         + select_theta_weight * theta_norm
 975:         + 2.00 * flat_penalty
 976:         + 2.00 * slope_penalty
 977:         + select_turn_t_weight * turn_t_penalty
 978:         + select_turn_left_weight * turn_left_penalty
 979:         + select_turn_lr_weight * turn_lr_penalty
 980:         + float(getattr(cfg, "select_theta_p95_weight", 0.0)) * theta_p95_penalty
 981:         + float(getattr(cfg, "select_theta_flat_p95_weight", 0.0)) * flat_p95_penalty
 982:         + float(getattr(cfg, "select_theta_near_flat_p95_weight", 0.0)) * near_flat_p95_penalty
 983:         + float(getattr(cfg, "select_theta_true_zero_p95_weight", 0.0)) * true_zero_p95_penalty
 984:         + float(getattr(cfg, "select_theta_flat_peak_weight", 0.0)) * flat_peak_penalty
 985:         + float(getattr(cfg, "select_theta_small_neg_p95_weight", 0.0)) * small_neg_p95_penalty
 986:         + float(getattr(cfg, "select_theta_extreme_p95_weight", 0.0)) * extreme_p95_penalty
 987:         + float(getattr(cfg, "select_theta_edge_p95_weight", 0.0)) * edge_p95_penalty
 988:         + float(getattr(cfg, "select_theta_small_nonzero_p95_weight", 0.0)) *
small_nonzero_p95_penalty
 989:         + float(getattr(cfg, "select_theta_flat_bias_weight", 0.0)) * flat_bias_penalty
 990:     )
 991:
 992:
 993: def seed42_gate(metrics: Dict[str, object], thresholds: GateThresholds = GateThresholds()) ->
Tuple[bool, List[str]]:
 994:     """判断 seed42 是否进入三 seed。"""
 995:
 996:     checks = [
 997:         ("acc_main", float(metrics["acc_main"]), ">=", thresholds.acc_main),
 998:         ("flat_recall", float(metrics["flat_recall"]), ">=", thresholds.flat_recall),
 999:         ("slope_recall", float(metrics["slope_recall"]), ">=", thresholds.slope_recall),
1000:         ("acc_turn_transition", float(metrics["acc_turn_transition"]), ">=",
thresholds.acc_turn_transition),
1001:         ("theta_mae_deg", float(metrics["theta_mae_deg"]), "<=", thresholds.theta_mae_deg),
1002:     ]
1003:     failed: List[str] = []
1004:     for name, value, op, threshold in checks:
1005:         passed = value >= threshold if op == ">=" else value <= threshold
1006:         if not passed:
1007:             failed.append(f"{name} {op} {threshold:.4f} 未满足，实际 {value:.4f}")
1008:     return len(failed) == 0, failed
1009:
1010:
1011: def _metric_excess_penalty(metrics: Dict[str, object], key: str, target: float, scale: float) ->
float:
1012:     value = float(metrics.get(key, float("nan")))
1013:     if np.isnan(value):
1014:         return 0.0
1015:     return max(0.0, value - target) / max(scale, 1e-6)
1016:
1017:
1018: def metric_row(seed: int, best_epoch: int, metrics: Dict[str, object], paths: Dict[str, str]) ->
Dict[str, object]:
1019:     return {
1020:         "model": "ModernTCN-small",
```

```
1021:         "seed": seed,
1022:         "best_epoch": best_epoch,
1023:         "acc_main": metrics["acc_main"],
1024:         "acc_turn": metrics["acc_turn"],
1025:         "acc_turn_pure": metrics["acc_turn_pure"],
1026:         "acc_turn_transition": metrics["acc_turn_transition"],
1027:         "main_confidence_mean": metrics.get("main_confidence_mean", float("nan")),
1028:         "main_low_conf_0p60_ratio": metrics.get("main_low_conf_0p60_ratio", float("nan")),
1029:         "main_low_conf_0p70_ratio": metrics.get("main_low_conf_0p70_ratio", float("nan")),
1030:         "turn_confidence_mean": metrics.get("turn_confidence_mean", float("nan")),
1031:         "turn_low_conf_0p60_ratio": metrics.get("turn_low_conf_0p60_ratio", float("nan")),
1032:         "turn_low_conf_0p70_ratio": metrics.get("turn_low_conf_0p70_ratio", float("nan")),
1033:         "turn_right_recall": metrics["turn_right_recall"],
1034:         "turn_straight_recall": metrics["turn_straight_recall"],
1035:         "turn_left_recall": metrics["turn_left_recall"],
1036:         "theta_mae_deg": metrics["theta_mae_deg"],
1037:         "theta_abs_le_8_mae_deg": metrics.get("theta_abs_le_8_mae_deg", float("nan")),
1038:         "theta_abs_le_8_rmse_deg": metrics.get("theta_abs_le_8_rmse_deg", float("nan")),
1039:         "theta_abs_le_8_p95_abs_err_deg": metrics.get("theta_abs_le_8_p95_abs_err_deg",
float("nan")),
1040:         "theta_abs_le_8_max_abs_err_deg": metrics.get("theta_abs_le_8_max_abs_err_deg",
float("nan")),
1041:         "theta_abs_le_8_bias_deg": metrics.get("theta_abs_le_8_bias_deg", float("nan")),
1042:         "theta_abs_le_10_mae_deg": metrics.get("theta_abs_le_10_mae_deg", float("nan")),
1043:         "theta_abs_le_10_rmse_deg": metrics.get("theta_abs_le_10_rmse_deg", float("nan")),
1044:         "theta_abs_le_10_p95_abs_err_deg": metrics.get("theta_abs_le_10_p95_abs_err_deg",
float("nan")),
1045:         "theta_abs_le_10_max_abs_err_deg": metrics.get("theta_abs_le_10_max_abs_err_deg",
float("nan")),
1046:         "theta_abs_le_10_bias_deg": metrics.get("theta_abs_le_10_bias_deg", float("nan")),
1047:         "theta_pos_8_10_mae_deg": metrics.get("theta_pos_8_10_mae_deg", float("nan")),
1048:         "theta_pos_8_10_rmse_deg": metrics.get("theta_pos_8_10_rmse_deg", float("nan")),
1049:         "theta_pos_8_10_p95_abs_err_deg": metrics.get("theta_pos_8_10_p95_abs_err_deg",
float("nan")),
1050:         "theta_pos_8_10_max_abs_err_deg": metrics.get("theta_pos_8_10_max_abs_err_deg",
float("nan")),
1051:         "theta_pos_8_10_bias_deg": metrics.get("theta_pos_8_10_bias_deg", float("nan")),
1052:         "theta_pos_8_10_n": metrics.get("theta_pos_8_10_n", float("nan")),
1053:         "theta_neg_10_8_mae_deg": metrics.get("theta_neg_10_8_mae_deg", float("nan")),
1054:         "theta_neg_10_8_rmse_deg": metrics.get("theta_neg_10_8_rmse_deg", float("nan")),
1055:         "theta_neg_10_8_p95_abs_err_deg": metrics.get("theta_neg_10_8_p95_abs_err_deg",
float("nan")),
1056:         "theta_neg_10_8_max_abs_err_deg": metrics.get("theta_neg_10_8_max_abs_err_deg",
float("nan")),
1057:         "theta_neg_10_8_bias_deg": metrics.get("theta_neg_10_8_bias_deg", float("nan")),
1058:         "theta_neg_10_8_n": metrics.get("theta_neg_10_8_n", float("nan")),
1059:         "theta_pos_6_8_mae_deg": metrics.get("theta_pos_6_8_mae_deg", float("nan")),
1060:         "theta_pos_6_8_rmse_deg": metrics.get("theta_pos_6_8_rmse_deg", float("nan")),
1061:         "theta_pos_6_8_p95_abs_err_deg": metrics.get("theta_pos_6_8_p95_abs_err_deg",
float("nan")),
1062:         "theta_pos_6_8_max_abs_err_deg": metrics.get("theta_pos_6_8_max_abs_err_deg",
float("nan")),
1063:         "theta_pos_6_8_bias_deg": metrics.get("theta_pos_6_8_bias_deg", float("nan")),
1064:         "theta_pos_6_8_n": metrics.get("theta_pos_6_8_n", float("nan")),
1065:         "theta_neg_8_6_mae_deg": metrics.get("theta_neg_8_6_mae_deg", float("nan")),
1066:         "theta_neg_8_6_rmse_deg": metrics.get("theta_neg_8_6_rmse_deg", float("nan")),
1067:         "theta_neg_8_6_p95_abs_err_deg": metrics.get("theta_neg_8_6_p95_abs_err_deg",
float("nan")),
1068:         "theta_neg_8_6_max_abs_err_deg": metrics.get("theta_neg_8_6_max_abs_err_deg",
float("nan")),
1069:         "theta_neg_8_6_bias_deg": metrics.get("theta_neg_8_6_bias_deg", float("nan")),
1070:         "theta_neg_8_6_n": metrics.get("theta_neg_8_6_n", float("nan")),
1071:         "theta_active_abs_ge_2_mae_deg": metrics.get("theta_active_abs_ge_2_mae_deg",
float("nan")),
1072:         "theta_active_abs_ge_2_p95_abs_err_deg":
metrics.get("theta_active_abs_ge_2_p95_abs_err_deg", float("nan")),
```

```
1073:         "theta_neg_4_2_mae_deg": metrics.get("theta_neg_4_2_mae_deg", float("nan")),
1074:         "theta_neg_4_2_p95_abs_err_deg": metrics.get("theta_neg_4_2_p95_abs_err_deg",
float("nan")),
1075:         "theta_neg_4_2_bias_deg": metrics.get("theta_neg_4_2_bias_deg", float("nan")),
1076:         "theta_neg_2_0p5_mae_deg": metrics.get("theta_neg_2_0p5_mae_deg", float("nan")),
1077:         "theta_neg_2_0p5_p95_abs_err_deg": metrics.get("theta_neg_2_0p5_p95_abs_err_deg",
float("nan")),
1078:         "theta_neg_2_0p5_bias_deg": metrics.get("theta_neg_2_0p5_bias_deg", float("nan")),
1079:         "theta_pos_0p5_2_mae_deg": metrics.get("theta_pos_0p5_2_mae_deg", float("nan")),
1080:         "theta_pos_0p5_2_p95_abs_err_deg": metrics.get("theta_pos_0p5_2_p95_abs_err_deg",
float("nan")),
```

```
1081:         "theta_pos_0p5_2_bias_deg": metrics.get("theta_pos_0p5_2_bias_deg", float("nan")),
1082:         "theta_flat_abs_p95_deg": metrics.get("theta_flat_abs_p95_deg", float("nan")),
1083:         "theta_flat_abs_max_deg": metrics.get("theta_flat_abs_max_deg", float("nan")),
1084:         "theta_flat_bias_deg": metrics.get("theta_flat_bias_deg", float("nan")),
1085:         "theta_near_flat_abs_p95_deg": metrics.get("theta_near_flat_abs_p95_deg", float("nan")),
1086:         "theta_near_flat_abs_max_deg": metrics.get("theta_near_flat_abs_max_deg", float("nan")),
1087:         "theta_near_flat_bias_deg": metrics.get("theta_near_flat_bias_deg", float("nan")),
1088:         "theta_true_zero_mae_deg": metrics.get("theta_true_zero_mae_deg", float("nan")),
1089:         "theta_true_zero_abs_p95_deg": metrics.get("theta_true_zero_abs_p95_deg", float("nan")),
1090:         "theta_true_zero_abs_max_deg": metrics.get("theta_true_zero_abs_max_deg", float("nan")),
1091:         "theta_true_zero_bias_deg": metrics.get("theta_true_zero_bias_deg", float("nan")),
1092:         "theta_flat_turn_abs_p95_deg": metrics.get("theta_flat_turn_abs_p95_deg", float("nan")),
1093:         "theta_flat_turn_abs_max_deg": metrics.get("theta_flat_turn_abs_max_deg", float("nan")),
1094:         "flat_recall": metrics["flat_recall"],
1095:         "stall_recall": metrics["stall_recall"],
1096:         "slope_recall": metrics["slope_recall"],
1097:         "uphill_recall": metrics["uphill_recall"],
1098:         "downhill_recall": metrics["downhill_recall"],
1099:         "checkpoint_file": paths.get("checkpoint_file", ""),
1100:         "onnx_file": paths.get("onnx_file", ""),
1101:         "report_file": paths.get("report_file", ""),
1102:     }
1103:
1104:
1105: def _weighted_ce(logits: torch.Tensor, labels: torch.Tensor, class_weights: torch.Tensor,
sample_weights: torch.Tensor) -> torch.Tensor:
1106:     class_weights = class_weights.to(logits.device)
1107:     per_sample = F.cross_entropy(logits, labels, weight=class_weights, reduction="none")
1108:     denom = (class_weights[labels] * sample_weights).sum().clamp_min(1e-8)
1109:     return (per_sample * sample_weights).sum() / denom
1110:
1111:
1112: def _masked_weighted_mse(pred: torch.Tensor, target: torch.Tensor, mask: torch.Tensor, weight:
torch.Tensor) -> torch.Tensor:
1113:     wm = mask * weight
1114:     return (((pred - target) ** 2) * wm).sum() / wm.sum().clamp_min(1e-8)
1115:
1116:
1117: def _masked_weighted_abs_excess_mse(
1118:     pred: torch.Tensor,
1119:     target: torch.Tensor,
1120:     mask: torch.Tensor,
1121:     weight: torch.Tensor,
1122:     target_deg: float,
1123: ) -> torch.Tensor:
1124:     wm = mask * weight
1125:     target_rad = torch.deg2rad(torch.tensor(float(target_deg), device=pred.device,
dtype=pred.dtype))
1126:     excess = torch.relu(torch.abs(pred - target) - target_rad)
1127:     return ((excess ** 2) * wm).sum() / wm.sum().clamp_min(1e-8)
1128:
1129:
1130: def _main_slope_sample_weight(y_main: torch.Tensor, theta: torch.Tensor, cfg) -> torch.Tensor:
1131:     w = torch.ones_like(theta)
1132:     w = torch.where((y_main == 2) & (theta < 0), torch.full_like(w, cfg.main_neg_slope_weight), w)
1133:     w = torch.where((y_main == 2) & (theta > 0), torch.full_like(w, cfg.main_pos_slope_weight), w)
1134:     return w
1135:
1136:
1137: def _theta_sign_weight(theta: torch.Tensor, cfg) -> torch.Tensor:
1138:     w = torch.ones_like(theta)
1139:     w = torch.where(theta < 0, torch.full_like(w, cfg.theta_neg_weight), w)
1140:     w = torch.where(theta > 0, torch.full_like(w, cfg.theta_pos_weight), w)
```

```
    return w


def _confusion_matrix(truth: np.ndarray, pred: np.ndarray, num_classes: int) -> np.ndarray:
    cm = np.zeros((num_classes, num_classes), dtype=np.int64)
    for t, p in zip(truth.reshape(-1), pred.reshape(-1)):
        if 0 <= int(t) < num_classes and 0 <= int(p) < num_classes:
            cm[int(t), int(p)] += 1
    return cm


def _softmax_np(logits: np.ndarray) -> np.ndarray:
    logits = np.asarray(logits, dtype=np.float64)
    logits = logits - np.max(logits, axis=1, keepdims=True)
    exp_logits = np.exp(logits)
    return exp_logits / np.maximum(exp_logits.sum(axis=1, keepdims=True), 1e-12)


def _confidence_summary(prefix: str, confidence: np.ndarray, correct: np.ndarray) -> Dict[str,
object]:
    confidence = np.asarray(confidence, dtype=np.float64).reshape(-1)
    correct = np.asarray(correct).reshape(-1).astype(bool)
    out: Dict[str, object] = {
        f"{prefix}_confidence_mean": float(np.mean(confidence)) if confidence.size else
float("nan"),
         f"{prefix}_confidence_error_mean": float(np.mean(confidence[~correct])) if np.any(~correct)
else float("nan"),
        f"{prefix}_low_conf_0p60_ratio": float(np.mean(confidence < 0.60)) if confidence.size else
float("nan"),
        f"{prefix}_low_conf_0p70_ratio": float(np.mean(confidence < 0.70)) if confidence.size else
float("nan"),
    }
    edges = np.array([0.0, 0.60, 0.70, 0.80, 0.90, 1.0000001], dtype=np.float64)
    bins = []
    for i in range(len(edges) - 1):
        lo = edges[i]
        hi = edges[i + 1]
        mask = (confidence >= lo) & (confidence < hi)
        n = int(mask.sum())
        bins.append(
            {
                "bin": f"[{lo:.2f},{min(hi, 1.0):.2f})",
                "n": n,
                "error_rate": float(np.mean(~correct[mask])) if n else float("nan"),
                "mean_confidence": float(np.mean(confidence[mask])) if n else float("nan"),
            }
        )
    out[f"{prefix}_confidence_bins"] = bins
    return out


def _masked_acc(pred: np.ndarray, truth: np.ndarray, mask: np.ndarray) -> float:
    mask = mask.astype(bool)
    if not np.any(mask):
        return float("nan")
    return float(np.mean(pred[mask] == truth[mask]))


def _slope_sub_recall(pred_main: np.ndarray, idx: Iterable[int]) -> float:
    idx = np.asarray(list(idx), dtype=np.int64)
    if idx.size == 0:
        return float("nan")
    return float(np.mean(pred_main[idx] == 2))


```

```
1201: def _theta_control_metrics(
1202:     theta_hat: np.ndarray,
1203:     theta_true: np.ndarray,
1204:     y_main: np.ndarray,
1205:     y_turn_raw: np.ndarray,
1206:     mask_theta: np.ndarray,
1207: ) -> Dict[str, float]:
1208:     """Metrics for whether theta_hat is safe enough to be used by scheduling."""
1209:
1210:     flat_mask = y_main == 0
1211:     near_flat_mask = np.abs(theta_true) <= np.deg2rad(0.5)
1212:     true_zero_mask = np.isclose(theta_true, 0.0, atol=np.deg2rad(1e-6))
1213:     flat_turn_mask = flat_mask & near_flat_mask & (y_turn_raw != 0)
1214:     slope_mask = np.asarray(mask_theta).reshape(-1).astype(bool)
1215:
1216:     out: Dict[str, float] = {}
1217:     out.update(_theta_zone("theta_flat", theta_hat, theta_true, flat_mask))
1218:     out.update(_theta_zone("theta_near_flat", theta_hat, theta_true, near_flat_mask))
1219:     out.update(_theta_zone("theta_true_zero", theta_hat, theta_true, true_zero_mask))
1220:     out.update(_theta_zone("theta_flat_turn", theta_hat, theta_true, flat_turn_mask))
1221:     out.update(_theta_zone("theta_slope_control", theta_hat, theta_true, slope_mask))
1222:     theta_deg = np.rad2deg(theta_true)
1223:     out.update(_theta_error_zone("theta_all", theta_hat, theta_true, slope_mask))
1224:     out.update(_theta_error_zone("theta_active_abs_ge_2", theta_hat, theta_true, slope_mask &
(np.abs(theta_deg) >= 2.0)))
1225:     out.update(_theta_error_zone("theta_abs_le_8", theta_hat, theta_true, slope_mask &
(np.abs(theta_deg) <= 8.0)))
1226:     out.update(_theta_error_zone("theta_abs_le_10", theta_hat, theta_true, slope_mask &
(np.abs(theta_deg) <= 10.0)))
1227:     out.update(_theta_error_zone("theta_pos_8_10", theta_hat, theta_true, slope_mask &
(theta_deg >= 8.0) & (theta_deg <= 10.0)))
1228:     out.update(_theta_error_zone("theta_neg_10_8", theta_hat, theta_true, slope_mask &
(theta_deg >= -10.0) & (theta_deg <= -8.0)))
1229:     out.update(_theta_error_zone("theta_pos_6_8", theta_hat, theta_true, slope_mask &
(theta_deg >= 6.0) & (theta_deg <= 8.0)))
1230:     out.update(_theta_error_zone("theta_neg_8_6", theta_hat, theta_true, slope_mask &
(theta_deg >= -8.0) & (theta_deg <= -6.0)))
1231:     out.update(_theta_error_zone("theta_neg_4_2", theta_hat, theta_true, slope_mask &
(theta_deg >= -4.0) & (theta_deg <= -2.0)))
1232:     out.update(_theta_error_zone("theta_neg_2_0p5", theta_hat, theta_true, slope_mask &
(theta_deg >= -2.0) & (theta_deg <= -0.5)))
1233:     out.update(_theta_error_zone("theta_pos_0p5_2", theta_hat, theta_true, slope_mask &
(theta_deg >= 0.5) & (theta_deg <= 2.0)))
1234:     return out
1235:
1236:
1237: def _theta_error_zone(prefix: str, theta_hat: np.ndarray, theta_true: np.ndarray, mask: np.ndarray)
-> Dict[str, float]:
1238:     mask = np.asarray(mask).reshape(-1).astype(bool)
1239:     if not np.any(mask):
1240:         return {
1241:             f"{prefix}_mae_deg": float("nan"),
1242:             f"{prefix}_rmse_deg": float("nan"),
1243:             f"{prefix}_p95_abs_err_deg": float("nan"),
1244:             f"{prefix}_max_abs_err_deg": float("nan"),
1245:             f"{prefix}_bias_deg": float("nan"),
1246:             f"{prefix}_n": 0.0,
1247:         }
1248:     err_deg = np.rad2deg(theta_hat[mask] - theta_true[mask])
1249:     return {
1250:         f"{prefix}_mae_deg": float(np.mean(np.abs(err_deg))),
1251:         f"{prefix}_rmse_deg": float(np.sqrt(np.mean(err_deg ** 2))),
1252:         f"{prefix}_p95_abs_err_deg": float(np.percentile(np.abs(err_deg), 95)),
1253:         f"{prefix}_max_abs_err_deg": float(np.max(np.abs(err_deg))),
1254:         f"{prefix}_bias_deg": float(np.mean(err_deg)),
1255:         f"{prefix}_n": float(np.count_nonzero(mask)),
```

```
1256:     }
1257:
1258:
1259: def _theta_zone(prefix: str, theta_hat: np.ndarray, theta_true: np.ndarray, mask: np.ndarray) ->
Dict[str, float]:
1260:     mask = np.asarray(mask).reshape(-1).astype(bool)
```

```
    if not np.any(mask):
        return {
            f"{prefix}_mae_deg": float("nan"),
            f"{prefix}_abs_p95_deg": float("nan"),
            f"{prefix}_abs_max_deg": float("nan"),
            f"{prefix}_bias_deg": float("nan"),
            f"{prefix}_n": 0.0,
        }
    err = theta_hat[mask] - theta_true[mask]
    abs_theta_deg = np.abs(np.rad2deg(theta_hat[mask]))
    return {
        f"{prefix}_mae_deg": float(np.rad2deg(np.mean(np.abs(err)))),
        f"{prefix}_abs_p95_deg": float(np.percentile(abs_theta_deg, 95)),
        f"{prefix}_abs_max_deg": float(np.max(abs_theta_deg)),
        f"{prefix}_bias_deg": float(np.rad2deg(np.mean(err))),
        f"{prefix}_n": float(np.count_nonzero(mask)),
    }

===== FILE: src/ModernTCN/train_modern_tcn.py =====
"""训练 ModernTCN-small，并输出与 TCN/GRU baseline 对齐的测试指标。

使用方式：
    python ModernTCN/train_modern_tcn.py --seed 42

重要约束：
    1. 默认读取 data/tcn/ModernTCN_dataset_v4_industrial.mat。
    2. 不重新划分 split，不重新拟合 scaler。
    3. 多 seed 实验建议通过 results/modern_tcn/scripts 下的脚本统一启动。
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import random
import time
from pathlib import Path
from typing import Dict, Iterable, Tuple

import numpy as np
import torch
from torch.utils.data import DataLoader

from modern_tcn_data import AGVWindowDataset, class_weights, find_project_root,
load_modern_tcn_dataset
from modern_tcn_metrics import compute_metrics, metric_row, multitask_loss, seed42_gate,
selection_score
from modern_tcn_model import ModernTCNConfig, ModernTCNSmall


def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(description="ModernTCN-small 第一阶段训练脚本")
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("--dataset-file", type=str, default="")
    p.add_argument("--run-tag", type=str, default="")
    p.add_argument("--epochs", type=int, default=120)
    p.add_argument("--batch-size", type=int, default=256)
    p.add_argument("--lr", type=float, default=1e-3)
    p.add_argument("--weight-decay", type=float, default=1e-4)
    p.add_argument("--patience", type=int, default=25)
```

```
    p.add_argument("--min-epochs", type=int, default=30)
    p.add_argument("--channels", type=int, default=64)
    p.add_argument("--blocks", type=int, default=5)
    p.add_argument("--kernel-size", type=int, default=31)
    p.add_argument(
        "--temporal-padding",
        type=str,
        default="same",
        choices=["same", "causal"],
        help="Temporal convolution padding mode. 默认 same，causal 仅用于因果消融实验。",
    )
    p.add_argument("--dropout", type=float, default=0.15)
    p.add_argument("--turn-head-source", type=str, default="full", choices=["full", "inputstats",
"kinematic_stats"])
    p.add_argument("--lambda-turn", type=float, default=0.05)
    p.add_argument("--lambda-theta", type=float, default=0.35)
    p.add_argument("--lambda-theta-flat", type=float, default=0.20)
    p.add_argument(
        "--theta-flat-loss-mode",
        type=str,
        default="near_zero",
        choices=["near_zero", "true_zero", "near_flat", "main_flat", "none"],
    )
    p.add_argument("--theta-flat-zero-tol-deg", type=float, default=0.3)
    p.add_argument("--lambda-theta-near-flat", type=float, default=0.0)
    p.add_argument("--theta-near-flat-deg", type=float, default=0.5)
    p.add_argument("--lambda-theta-error-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-flat-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-near-flat-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-true-zero-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-active-excess", type=float, default=0.0)
    p.add_argument("--lambda-theta-small-neg", type=float, default=0.0)
    p.add_argument("--lambda-theta-small-neg-excess", type=float, default=0.0)
    p.add_argument("--theta-excess-target-deg", type=float, default=1.0)
    p.add_argument("--theta-flat-excess-target-deg", type=float, default=0.5)
    p.add_argument("--theta-small-neg-min-deg", type=float, default=-4.0)
    p.add_argument("--theta-small-neg-max-deg", type=float, default=-2.0)
    p.add_argument("--theta-gate-mode", type=str, default="none", choices=["none",
"main_slope_prob"])
    p.add_argument("--theta-gate-power", type=float, default=1.0)
    p.add_argument("--theta-gate-floor", type=float, default=0.0)
    p.add_argument("--theta-neg-weight", type=float, default=1.0)
    p.add_argument("--theta-pos-weight", type=float, default=1.0)
    p.add_argument("--turn-transition-weight", type=float, default=1.0)
    p.add_argument("--turn-class-multipliers", type=float, nargs=3, default=[1.00, 1.10, 1.00])
    p.add_argument("--select-turn-weight", type=float, default=0.30)
    p.add_argument("--select-turn-transition-weight", type=float, default=1.00)
    p.add_argument("--select-turn-transition-target", type=float, default=0.75)
    p.add_argument("--select-turn-left-weight", type=float, default=0.00)
    p.add_argument("--select-turn-left-target", type=float, default=0.80)
    p.add_argument("--select-turn-lr-weight", type=float, default=0.00)
    p.add_argument("--select-turn-lr-target", type=float, default=0.80)
    p.add_argument("--select-theta-weight", type=float, default=0.15)
    p.add_argument("--select-theta-ref-deg", type=float, default=5.0)
    p.add_argument("--select-theta-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-flat-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-flat-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-near-flat-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-near-flat-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-true-zero-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-true-zero-p95-target-deg", type=float, default=1.0)
```

```
    p.add_argument("--select-theta-extreme-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-extreme-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-edge-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-edge-p95-target-deg", type=float, default=1.2)
    p.add_argument("--select-theta-small-nonzero-p95-weight", type=float, default=0.0)
    p.add_argument("--select-theta-small-nonzero-p95-target-deg", type=float, default=1.0)
    p.add_argument("--select-theta-flat-bias-weight", type=float, default=0.0)
    p.add_argument("--select-theta-flat-bias-target-deg", type=float, default=0.2)
    p.add_argument("--device", type=str, default="auto", choices=["auto", "cpu", "cuda"])
    p.add_argument("--num-workers", type=int, default=0)
    p.add_argument("--limit-train", type=int, default=0, help="仅用于 smoke test，正式实验必须为 0。
")
    p.add_argument("--limit-val", type=int, default=0, help="仅用于 smoke test，正式实验必须为 0。")
    p.add_argument("--limit-test", type=int, default=0, help="仅用于 smoke test，正式实验必须为 0。")
    p.add_argument("--dry-run", action="store_true", help="只读取数据并跑一次前向，不保存训练结果。")
    return p.parse_args()


def main() -> Dict[str, object]:
    args = parse_args()
    return train_one_seed(args)


def train_one_seed(args: argparse.Namespace) -> Dict[str, object]:
    root = find_project_root()
    run_tag = args.run_tag or f"modern_tcn_v4_industrial_seed{args.seed}"
    out_dir = root / "results" / "modern_tcn" / run_tag
    dataset_file = Path(args.dataset_file) if args.dataset_file else None

    _set_seed(args.seed)
    device = _select_device(args.device)
    data = load_modern_tcn_dataset(
        dataset_file=dataset_file,
        limit_train=args.limit_train,
        limit_val=args.limit_val,
        limit_test=args.limit_test,
    )
    train_split = data["train"]
    val_split = data["val"]
    test_split = data["test"]
    contract = data["contract"]

    cfg = ModernTCNConfig(
        input_dim=contract.input_dim,
        seq_len=contract.seq_len,
        channels=args.channels,
        blocks=args.blocks,
        kernel_size=args.kernel_size,
        temporal_padding=getattr(args, "temporal_padding", "same"),
        dropout=args.dropout,
        turn_head_source=args.turn_head_source,
        lambda_turn=args.lambda_turn,
        lambda_theta=args.lambda_theta,
        lambda_theta_flat=args.lambda_theta_flat,
        theta_flat_loss_mode=args.theta_flat_loss_mode,
        theta_flat_zero_tol_deg=args.theta_flat_zero_tol_deg,
        lambda_theta_near_flat=args.lambda_theta_near_flat,
        theta_near_flat_deg=args.theta_near_flat_deg,
        lambda_theta_error_excess=args.lambda_theta_error_excess,
        lambda_theta_flat_excess=args.lambda_theta_flat_excess,
        lambda_theta_near_flat_excess=args.lambda_theta_near_flat_excess,
```

```
1441:         lambda_theta_true_zero_excess=args.lambda_theta_true_zero_excess,
1442:         lambda_theta_active_excess=args.lambda_theta_active_excess,
1443:         lambda_theta_small_neg=args.lambda_theta_small_neg,
1444:         lambda_theta_small_neg_excess=args.lambda_theta_small_neg_excess,
1445:         theta_excess_target_deg=args.theta_excess_target_deg,
1446:         theta_flat_excess_target_deg=args.theta_flat_excess_target_deg,
1447:         theta_small_neg_min_deg=args.theta_small_neg_min_deg,
1448:         theta_small_neg_max_deg=args.theta_small_neg_max_deg,
1449:         theta_gate_mode=args.theta_gate_mode,
1450:         theta_gate_power=args.theta_gate_power,
1451:         theta_gate_floor=args.theta_gate_floor,
1452:         theta_neg_weight=args.theta_neg_weight,
1453:         theta_pos_weight=args.theta_pos_weight,
1454:         turn_transition_weight=args.turn_transition_weight,
1455:         turn_class_multipliers=tuple(args.turn_class_multipliers),
1456:         select_turn_weight=args.select_turn_weight,
1457:         select_turn_transition_weight=args.select_turn_transition_weight,
1458:         select_turn_transition_target=getattr(args, "select_turn_transition_target", 0.75),
1459:         select_turn_left_weight=args.select_turn_left_weight,
1460:         select_turn_left_target=args.select_turn_left_target,
1461:         select_turn_lr_weight=getattr(args, "select_turn_lr_weight", 0.0),
1462:         select_turn_lr_target=getattr(args, "select_turn_lr_target", 0.80),
1463:         select_theta_weight=args.select_theta_weight,
1464:         select_theta_ref_deg=args.select_theta_ref_deg,
1465:         select_theta_p95_weight=args.select_theta_p95_weight,
1466:         select_theta_p95_target_deg=getattr(args, "select_theta_p95_target_deg", 1.0),
1467:         select_theta_flat_p95_weight=args.select_theta_flat_p95_weight,
1468:         select_theta_flat_p95_target_deg=getattr(args, "select_theta_flat_p95_target_deg", 1.0),
1469:         select_theta_near_flat_p95_weight=args.select_theta_near_flat_p95_weight,
1470:         select_theta_near_flat_p95_target_deg=getattr(args,
"select_theta_near_flat_p95_target_deg", 1.0),
1471:         select_theta_true_zero_p95_weight=args.select_theta_true_zero_p95_weight,
1472:         select_theta_true_zero_p95_target_deg=getattr(args,
"select_theta_true_zero_p95_target_deg", 1.0),
1473:         select_theta_extreme_p95_weight=args.select_theta_extreme_p95_weight,
1474:         select_theta_extreme_p95_target_deg=getattr(args, "select_theta_extreme_p95_target_deg",
1.0),
1475:         select_theta_edge_p95_weight=getattr(args, "select_theta_edge_p95_weight", 0.0),
1476:         select_theta_edge_p95_target_deg=getattr(args, "select_theta_edge_p95_target_deg", 1.2),
1477:         select_theta_small_nonzero_p95_weight=args.select_theta_small_nonzero_p95_weight,
1478:         select_theta_small_nonzero_p95_target_deg=getattr(args,
"select_theta_small_nonzero_p95_target_deg", 1.0),
1479:         select_theta_flat_bias_weight=args.select_theta_flat_bias_weight,
1480:         select_theta_flat_bias_target_deg=getattr(args, "select_theta_flat_bias_target_deg", 0.2),
1481:     )
1482:     model = ModernTCNSmall(cfg).to(device)
1483:
1484:     # smoke test 只验证数据契约和模型维度，不写任何模型文件。
1485:     if args.dry_run:
1486:         xb = torch.from_numpy(train_split.X[:4]).float().to(device)
1487:         with torch.no_grad():
1488:             outputs = model(xb)
1489:         print("[ModernTCN dry-run] 数据和模型前向检查通过")
1490:         print(f"  X: {tuple(xb.shape)}")
1491:         print(f"  logits_main/logits_turn/theta: {[tuple(o.shape) for o in outputs]}")
1492:         return {"status": "dry_run_ok"}
1493:
1494:     out_dir.mkdir(parents=True, exist_ok=True)
1495:     checkpoint_file = out_dir / f"modern_tcn_seed{args.seed}.pt"
1496:     summary_csv = out_dir / f"modern_tcn_seed{args.seed}_summary.csv"
1497:     history_csv = out_dir / f"modern_tcn_seed{args.seed}_history.csv"
1498:     report_file = out_dir / "ModernTCN_train_report.md"
1499:
1500:     train_loader = DataLoader(
```

```
        AGVWindowDataset(train_split),
        batch_size=args.batch_size,
        shuffle=True,
        num_workers=args.num_workers,
        pin_memory=(device.type == "cuda"),
    )
    val_loader = DataLoader(AGVWindowDataset(val_split), batch_size=args.batch_size, shuffle=False, num_workers=0)
    test_loader = DataLoader(AGVWindowDataset(test_split), batch_size=args.batch_size, shuffle=False, num_workers=0)

    class_w_main = class_weights(
        train_split.y_main,
        3,
        cfg.main_class_weight_method,
        list(cfg.main_class_multipliers),
    ).to(device)
    class_w_turn = class_weights(
        train_split.y_turn,
        3,
        cfg.turn_class_weight_method,
        list(cfg.turn_class_multipliers),
    ).to(device)

    opt = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=max(args.epochs, 1))

    print("[ModernTCN] 第一阶段训练开始")
    print(f"  seed={args.seed}, device={device}, out={out_dir}")
    print(f"  dataset={contract.dataset_file}")
    print(f"  train/val/test={len(train_split.X)}/{len(val_split.X)}/{len(test_split.X)}")
    print(
        f"  model channels={cfg.channels}, blocks={cfg.blocks}, kernel={cfg.kernel_size}, "
        f"temporal_padding={cfg.temporal_padding}"
    )

    best_score = math.inf
    best_epoch = 0
    best_state = None
    best_val_metrics: Dict[str, object] = {}
    patience_count = 0
    history_rows = []
    t0 = time.time()

    for epoch in range(1, args.epochs + 1):
        train_loss = _train_epoch(model, train_loader, opt, device, class_w_main, class_w_turn, cfg)
        scheduler.step()
        val_loss, val_logits_main, val_logits_turn, val_theta = _predict_full(
            model, val_loader, val_split, device, class_w_main, class_w_turn, cfg
        )
        val_metrics = compute_metrics(val_logits_main, val_logits_turn, val_theta, val_split, val_loss)
        score = selection_score(val_metrics, cfg)
        history_rows.append(_history_row(epoch, opt.param_groups[0]["lr"], train_loss, val_metrics, score))

        if score < best_score:
            best_score = score
            best_epoch = epoch
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
            best_val_metrics = dict(val_metrics)
            patience_count = 0
        else:
            patience_count += 1
```

```
1561:
1562:         if epoch == 1 or epoch % 5 == 0 or epoch == args.epochs:
1563:             print(
1564:                 f"  epoch {epoch:03d} | train={train_loss:.4f} val={val_metrics['loss_total']:.4f}
"
1565:                 f"main={val_metrics['acc_main']:.4f} turn={val_metrics['acc_turn']:.4f} "
1566:                 f"turnL={val_metrics['turn_left_recall']:.4f}
turnT={val_metrics['acc_turn_transition']:.4f} "
1567:                 f"theta={val_metrics['theta_mae_deg']:.4f} score={score:.4f}"
1568:             )
1569:
1570:         if epoch >= args.min_epochs and patience_count >= args.patience:
1571:             print(f"[ModernTCN] 早停：epoch={epoch}, best_epoch={best_epoch}")
1572:             break
1573:
1574:     if best_state is None:
1575:         raise RuntimeError("训练未产生有效 checkpoint。")
1576:
1577:     model.load_state_dict(best_state)
1578:     test_loss, logits_main, logits_turn, theta_hat = _predict_full(
1579:         model, test_loader, test_split, device, class_w_main, class_w_turn, cfg
1580:     )
1581:     test_metrics = compute_metrics(logits_main, logits_turn, theta_hat, test_split, test_loss)
1582:     train_seconds = time.time() - t0
1583:
1584:     torch.save(
1585:         {
1586:             "model_state": best_state,
1587:             "model_config": cfg.to_dict(),
1588:             "seed": args.seed,
1589:             "best_epoch": best_epoch,
1590:             "best_val_score": best_score,
1591:             "best_val_metrics": best_val_metrics,
1592:             "test_metrics": test_metrics,
1593:             "contract": contract.__dict__,
1594:             "feat_names": data["feat_names"],
1595:             "scaler": data["scaler"],
1596:             "train_seconds": train_seconds,
1597:         },
1598:         checkpoint_file,
1599:     )
1600:
1601:     paths = {"checkpoint_file": str(checkpoint_file), "report_file": str(report_file)}
1602:     row = metric_row(args.seed, best_epoch, test_metrics, paths)
1603:     _write_csv(summary_csv, [row])
1604:     _write_csv(history_csv, history_rows)
1605:     _write_report(report_file, args, cfg, contract.__dict__, row, test_metrics, best_val_metrics,
train_seconds)
1606:
1607:     print("[ModernTCN] 训练完成")
1608:     print(f"  checkpoint: {checkpoint_file}")
1609:     print(f"  summary: {summary_csv}")
1610:     print(f"  report: {report_file}")
1611:     print(
1612:         f"  test main={test_metrics['acc_main']:.4f},
turnT={test_metrics['acc_turn_transition']:.4f}, "
1613:         f"turnL={test_metrics['turn_left_recall']:.4f}, theta={test_metrics['theta_mae_deg']:.4f},
flat={test_metrics['flat_recall']:.4f}, "
1614:         f"slope={test_metrics['slope_recall']:.4f}"
1615:     )
1616:
1617:     if args.seed == 42:
1618:         passed, failures = seed42_gate(test_metrics)
1619:         print(f"  seed42 gate pass={int(passed)}")
1620:         for msg in failures:
```

```
1621:             print(f"    - {msg}")
1622:
1623:     return {
1624:         "checkpoint_file": str(checkpoint_file),
1625:         "summary_csv": str(summary_csv),
1626:         "report_file": str(report_file),
1627:         "test_metrics": test_metrics,
1628:         "best_epoch": best_epoch,
1629:     }
1630:
1631:
1632: def _train_epoch(model, loader, opt, device, class_w_main, class_w_turn, cfg) -> float:
1633:     model.train()
1634:     total_loss = 0.0
1635:     total_n = 0
1636:     for batch in loader:
1637:         batch = _to_device(batch, device)
1638:         opt.zero_grad(set_to_none=True)
1639:         logits_main, logits_turn, theta_hat = model(batch["X"])
1640:         loss, _ = multitask_loss(logits_main, logits_turn, theta_hat, batch, class_w_main,
class_w_turn, cfg)
1641:         loss.backward()
1642:         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
1643:         opt.step()
1644:         n = int(batch["X"].shape[0])
1645:         total_loss += float(loss.detach().cpu()) * n
1646:         total_n += n
1647:     return total_loss / max(total_n, 1)
1648:
1649:
1650: @torch.no_grad()
1651: def _predict_full(model, loader, split, device, class_w_main, class_w_turn, cfg) -> Tuple[float,
np.ndarray, np.ndarray, np.ndarray]:
1652:     model.eval()
1653:     logits_main_all = []
1654:     logits_turn_all = []
1655:     theta_all = []
1656:     loss_sum = 0.0
1657:     n_sum = 0
1658:     for batch in loader:
1659:         batch = _to_device(batch, device)
1660:         logits_main, logits_turn, theta_hat = model(batch["X"])
1661:         loss, _ = multitask_loss(logits_main, logits_turn, theta_hat, batch, class_w_main,
class_w_turn, cfg)
1662:         n = int(batch["X"].shape[0])
1663:         loss_sum += float(loss.detach().cpu()) * n
1664:         n_sum += n
1665:         logits_main_all.append(logits_main.detach().cpu().numpy())
1666:         logits_turn_all.append(logits_turn.detach().cpu().numpy())
1667:         theta_all.append(theta_hat.detach().cpu().numpy())
1668:     return (
1669:         loss_sum / max(n_sum, 1),
1670:         np.concatenate(logits_main_all, axis=0),
1671:         np.concatenate(logits_turn_all, axis=0),
1672:         np.concatenate(theta_all, axis=0).reshape(-1),
1673:     )
1674:
1675:
1676: def _to_device(batch: Dict[str, torch.Tensor], device: torch.device) -> Dict[str, torch.Tensor]:
1677:     return {k: v.to(device, non_blocking=True) for k, v in batch.items()}
1678:
1679:
1680: def _set_seed(seed: int) -> None:
```

```
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True


def _select_device(mode: str) -> torch.device:
    if mode == "cuda":
        return torch.device("cuda")
    if mode == "cpu":
        return torch.device("cpu")
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


def _history_row(epoch: int, lr: float, train_loss: float, val_metrics: Dict[str, object], score:
float) -> Dict[str, object]:
    return {
        "epoch": epoch,
        "lr": lr,
        "train_loss": train_loss,
        "val_loss": val_metrics["loss_total"],
        "val_score": score,
        "val_acc_main": val_metrics["acc_main"],
        "val_acc_turn": val_metrics["acc_turn"],
        "val_main_confidence_mean": val_metrics.get("main_confidence_mean", float("nan")),
        "val_turn_confidence_mean": val_metrics.get("turn_confidence_mean", float("nan")),
        "val_main_low_conf_0p60_ratio": val_metrics.get("main_low_conf_0p60_ratio", float("nan")),
        "val_turn_low_conf_0p60_ratio": val_metrics.get("turn_low_conf_0p60_ratio", float("nan")),
        "val_turn_left_recall": val_metrics["turn_left_recall"],
        "val_turn_right_recall": val_metrics["turn_right_recall"],
        "val_acc_turn_transition": val_metrics["acc_turn_transition"],
        "val_theta_mae_deg": val_metrics["theta_mae_deg"],
        "val_theta_abs_le_10_p95_abs_err_deg": val_metrics["theta_abs_le_10_p95_abs_err_deg"],
        "val_theta_neg_10_8_p95_abs_err_deg": val_metrics["theta_neg_10_8_p95_abs_err_deg"],
        "val_theta_pos_8_10_p95_abs_err_deg": val_metrics["theta_pos_8_10_p95_abs_err_deg"],
        "val_theta_abs_le_8_p95_abs_err_deg": val_metrics["theta_abs_le_8_p95_abs_err_deg"],
        "val_theta_neg_8_6_p95_abs_err_deg": val_metrics["theta_neg_8_6_p95_abs_err_deg"],
        "val_theta_pos_6_8_p95_abs_err_deg": val_metrics["theta_pos_6_8_p95_abs_err_deg"],
        "val_theta_neg_2_0p5_p95_abs_err_deg": val_metrics["theta_neg_2_0p5_p95_abs_err_deg"],
        "val_theta_pos_0p5_2_p95_abs_err_deg": val_metrics["theta_pos_0p5_2_p95_abs_err_deg"],
        "val_theta_flat_abs_p95_deg": val_metrics["theta_flat_abs_p95_deg"],
        "val_theta_flat_bias_deg": val_metrics["theta_flat_bias_deg"],
        "val_theta_near_flat_abs_p95_deg": val_metrics["theta_near_flat_abs_p95_deg"],
        "val_theta_true_zero_abs_p95_deg": val_metrics["theta_true_zero_abs_p95_deg"],
        "val_flat_recall": val_metrics["flat_recall"],
        "val_stall_recall": val_metrics["stall_recall"],
        "val_slope_recall": val_metrics["slope_recall"],
    }


def _write_csv(path: Path, rows: Iterable[Dict[str, object]]) -> None:
    rows = list(rows)
    if not rows:
        return
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

```

```
1741:
1742: def _write_report(
1743:     path: Path,
1744:     args: argparse.Namespace,
1745:     cfg: ModernTCNConfig,
1746:     contract: Dict[str, object],
1747:     row: Dict[str, object],
1748:     test_metrics: Dict[str, object],
1749:     val_metrics: Dict[str, object],
1750:     train_seconds: float,
1751: ) -> None:
1752:     passed, failures = seed42_gate(test_metrics) if args.seed == 42 else (False, [])
1753:     with path.open("w", encoding="utf-8") as f:
1754:         f.write("# ModernTCN-small 第一阶段训练报告\n\n")
1755:         f.write("## 固定约束\n\n")
1756:         f.write(f"- dataset: `{contract['dataset_file']}`\n")
1757:         f.write(f"- vehicle: `{contract.get('vehicle_type', '')}`;
active=`{contract.get('active_drive_steer_wheels', '')}`;
passive=`{contract.get('passive_support_wheels', '')}`\n")
1758:         f.write(f"- feature_policy: `{contract.get('feature_policy', '')}`\n")
1759:         f.write(f"- label_time_policy: `{contract.get('label_time_policy', '')}`,
horizon_steps={contract.get('horizon_steps', 0)}\n")
1760:         f.write("- split: 使用 MAT 文件已有 run-level split，不重划分。\n")
1761:         f.write("- scaler: 使用 MAT 文件已有归一化后 X，不重新拟合。\n")
1762:         f.write(f"- confidence_policy: `{contract.get('confidence_policy', '')}`\n")
1763:         f.write(f"- input: `[batch, time={contract['seq_len']},
feature={contract['input_dim']}]`\n")
1764:         f.write("- output: `logits_main`, `logits_turn`, `theta_hat`\n\n")
1765:         f.write("## 配置\n\n")
1766:         f.write("```json\n")
1767:         f.write(json.dumps(cfg.to_dict(), indent=2, ensure_ascii=False))
1768:         f.write("\n```\n\n")
1769:         f.write("## 测试集指标\n\n")
1770:         f.write("| metric | value |\n|---|---:|\n")
1771:         for key in [
1772:             "acc_main",
1773:             "acc_turn",
1774:             "acc_turn_pure",
1775:             "acc_turn_transition",
1776:             "main_confidence_mean",
1777:             "main_low_conf_0p60_ratio",
1778:             "main_low_conf_0p70_ratio",
1779:             "turn_confidence_mean",
1780:             "turn_low_conf_0p60_ratio",
1781:             "turn_low_conf_0p70_ratio",
1782:             "turn_right_recall",
1783:             "turn_straight_recall",
1784:             "turn_left_recall",
1785:             "theta_mae_deg",
1786:             "theta_abs_le_10_p95_abs_err_deg",
1787:             "theta_neg_10_8_p95_abs_err_deg",
1788:             "theta_pos_8_10_p95_abs_err_deg",
1789:             "theta_abs_le_8_p95_abs_err_deg",
1790:             "theta_neg_8_6_p95_abs_err_deg",
1791:             "theta_pos_6_8_p95_abs_err_deg",
1792:             "theta_neg_2_0p5_p95_abs_err_deg",
1793:             "theta_pos_0p5_2_p95_abs_err_deg",
1794:             "theta_flat_abs_p95_deg",
1795:             "theta_flat_bias_deg",
1796:             "theta_near_flat_abs_p95_deg",
1797:             "theta_true_zero_abs_p95_deg",
1798:             "theta_near_flat_bias_deg",
1799:             "theta_flat_turn_abs_p95_deg",
1800:             "flat_recall",
```

```
1801:
1802: ===== [中间省略 28 页] =====
1803:
1804: fid = fopen(md_file, 'w');
1805: if fid < 0
1806:     warning('ModernTCN:ReportFailed', '无法写入报告: %s', md_file);
1807:     return;
1808: end
1809: cleanup = onCleanup(@() fclose(fid));
1810: fprintf(fid, '# ModernTCN MATLAB ONNX 一致性检查\n\n');
1811: fprintf(fid, '- onnx: `%s`\n', result.onnx_file);
1812: fprintf(fid, '- sample: `%s`\n', result.sample_file);
1813: fprintf(fid, '- pass: `%d`\n\n', result.pass);
1814: fprintf(fid, '| output | max abs error | mean abs error |\n');
1815: fprintf(fid, '|---|---:|---:|\n');
1816: fprintf(fid, '| logits_main | %.6g | %.6g |\n', ...
1817:     result.logits_main.max_abs_error, result.logits_main.mean_abs_error);
1818: fprintf(fid, '| logits_turn | %.6g | %.6g |\n', ...
1819:     result.logits_turn.max_abs_error, result.logits_turn.mean_abs_error);
1820: fprintf(fid, '| theta_hat | %.6g | %.6g |\n', ...
1821:     result.theta_hat.max_abs_error, result.theta_hat.mean_abs_error);
1822: end
1823:
1824: ===== FILE: src/core/preloadfcn_modern_tcn.m =====
1825: function preloadfcn_modern_tcn()
1826: %PRELOADFCN_MODERN_TCN PreLoadFcn entry for the ModernTCN closed-loop model.
1827: %
1828: % The common LPV/MPC initialization lives in preloadfcn_gru. The mode flag
1829: % selects the frozen ModernTCN artifact instead of loading a GRU artifact.
1830:
1831: preloadfcn_gru('modern_tcn');
1832: end
1833:
1834: ===== FILE: src/ModernTCN/ModernTCN_analyze_closed_loop_out.m =====
1835: function result = ModernTCN_analyze_closed_loop_out(out_file)
1836: %MODERNTCN_ANALYZE_CLOSED_LOOP_OUT Analyze ModernTCN closed-loop Simulink logs.
1837: %
1838: %   result = ModernTCN_analyze_closed_loop_out('out.mat')
1839: %
1840: % The analysis expects logsout to contain diag.y_raw plus the public diag.*
1841: % signals from LPVMPC_AGV_simulink_Modern_TCN. It does not modify the model.
1842:
1843: if nargin < 1 || isempty(out_file)
1844:     out_file = fullfile(local_project_root(), 'out.mat');
1845: end
1846: if exist('init_project', 'file') == 2
1847:     init_project();
1848: end
1849:
1850: root = local_project_root();
1851: S = load(out_file, 'logsout');
1852: logs = S.logsout;
1853:
1854: t = local_time(logs, 'diag.y_raw');
1855: y_raw = local_data(logs, 'diag.y_raw');
1856: theta_hat = local_resample(logs, 'diag.theta_hat', t);
1857: label_main = local_resample(logs, 'diag.label_main', t);
1858: label_turn = local_resample(logs, 'diag.label_turn', t);
1859: conf_main = local_resample(logs, 'diag.conf_main', t);
1860: theta_ground = local_resample(logs, 'diag.theta_ground', t);
```

```
theta_ref = local_resample(logs, 'diag.theta_ref', t);
omega_ref = local_resample(logs, 'diag.omega_ref', t);
v_ref = local_resample(logs, 'diag.v_ref', t);

e_y = local_resample(logs, 'diag.e_y', t);
e_psi = local_resample(logs, 'diag.e_psi', t);
e_v = local_resample(logs, 'diag.e_v', t);
e_omega = local_resample(logs, 'diag.e_omega', t);
F_cmd = local_resample(logs, 'diag.F_cmd', t);
omega_cmd = local_resample(logs, 'diag.omega_cmd', t);

default_cfg = ModernTCN_default_config(root);
dataset_file = default_cfg.dataset_file;
Ds = load(dataset_file, 'dataset');
dataset = Ds.dataset;

params = parameters();
[feature_raw, feature_norm] = local_extract_features_from_yraw(y_raw, t, params, dataset);

zones = local_get_zones(root, t);
zone_names = fieldnames(zones);

truth = local_make_truth(theta_ground, omega_ref);

rows = repmat(local_empty_zone_row(), numel(zone_names), 1);
for i = 1:numel(zone_names)
    name = zone_names{i};
    tr = zones.(name);
    mask = t >= tr(1) & t < tr(2);
    rows(i) = local_zone_row(name, tr, mask, t, e_y, e_psi, e_v, e_omega, ...
        theta_hat, theta_ground, label_main, label_turn, conf_main, truth, ...
        F_cmd, omega_cmd);
end
zone_table = struct2table(rows);

feature_zone = local_feature_zone_summary(zone_names, zones, t, feature_norm, dataset.feat_names);
feature_compare = local_feature_compare_to_training(dataset, feature_norm, t, zones);
assessment = local_assess_closed_loop(zone_table, feature_zone);

out_dir = fullfile(root, 'results', 'modern_tcn', 'closed_loop_diag');
if exist(out_dir, 'dir') ~= 7
    mkdir(out_dir);
end
[~, base_name, ~] = fileparts(out_file);
mat_file = fullfile(out_dir, sprintf('%s_modern_tcn_closed_loop_diag.mat', base_name));
report_file = fullfile(out_dir, sprintf('%s_modern_tcn_closed_loop_diag_report.md', base_name));

result = struct();
result.out_file = out_file;
result.dataset_file = dataset_file;
result.zone_table = zone_table;
result.feature_zone = feature_zone;
result.feature_compare = feature_compare;
result.assessment = assessment;
result.feature_names = dataset.feat_names(:);
result.theta_error_deg = rad2deg(theta_hat - theta_ground);
result.report_file = report_file;
result.mat_file = mat_file;

save(mat_file, 'result');
```

```
local_write_report(report_file, result);

fprintf('[ModernTCN closed-loop diag] report: %s\n', report_file);
fprintf('[ModernTCN closed-loop diag] mat   : %s\n', mat_file);
disp(zone_table(:, {'zone','ey_rmse','epsi_rmse','ev_rmse','theta_mae_deg', ...
    'main_acc_pct','turn_acc_pct','pred_main_1_pct','pred_main_3_pct', ...
    'pred_turn_0_pct','pred_turn_1_pct'}));
disp(assessment(:, {'check','value','threshold','pass'}));
end

function row = local_empty_zone_row()
row = struct('zone', "", 't0', NaN, 't1', NaN, ...
    'ey_rmse', NaN, 'ey_peak', NaN, 'epsi_rmse', NaN, 'epsi_peak', NaN, ...
    'ev_rmse', NaN, 'ev_peak', NaN, 'eomega_rmse', NaN, 'eomega_peak', NaN, ...
    'theta_mae_deg', NaN, 'theta_rmse_deg', NaN, 'theta_peak_deg', NaN, ...
    'theta_hat_min_deg', NaN, 'theta_hat_max_deg', NaN, ...
    'main_acc_pct', NaN, 'turn_acc_pct', NaN, ...
    'true_main_1_pct', NaN, 'true_main_3_pct', NaN, ...
    'pred_main_1_pct', NaN, 'pred_main_2_pct', NaN, 'pred_main_3_pct', NaN, ...
    'true_turn_m1_pct', NaN, 'true_turn_0_pct', NaN, 'true_turn_1_pct', NaN, ...
    'pred_turn_m1_pct', NaN, 'pred_turn_0_pct', NaN, 'pred_turn_1_pct', NaN, ...
    'conf_median', NaN, 'conf_p10', NaN, ...
    'F_min', NaN, 'F_max', NaN, 'omega_cmd_min', NaN, 'omega_cmd_max', NaN);
end

function row = local_zone_row(name, tr, mask, ~, e_y, e_psi, e_v, e_omega, ...
    theta_hat, theta_ground, label_main, label_turn, conf_main, truth, F_cmd, omega_cmd)
row = local_empty_zone_row();
row.zone = string(name);
row.t0 = tr(1);
row.t1 = tr(2);
if nnz(mask) < 2
    return;
end

th_err = theta_hat(mask) - theta_ground(mask);
row.ey_rmse = local_rms(e_y(mask));
row.ey_peak = max(abs(e_y(mask)));
row.epsi_rmse = local_rms(e_psi(mask));
row.epsi_peak = max(abs(e_psi(mask)));
row.ev_rmse = local_rms(e_v(mask));
row.ev_peak = max(abs(e_v(mask)));
row.eomega_rmse = local_rms(e_omega(mask));
row.eomega_peak = max(abs(e_omega(mask)));
row.theta_mae_deg = rad2deg(mean(abs(th_err), 'omitnan'));
row.theta_rmse_deg = rad2deg(local_rms(th_err));
row.theta_peak_deg = rad2deg(max(abs(th_err)));
row.theta_hat_min_deg = rad2deg(min(theta_hat(mask)));
row.theta_hat_max_deg = rad2deg(max(theta_hat(mask)));

lm = round(label_main(mask));
lt = round(label_turn(mask));
tm = truth.main(mask);
tt = truth.turn(mask);
row.main_acc_pct = 100 * mean(lm == tm, 'omitnan');
row.turn_acc_pct = 100 * mean(lt == tt, 'omitnan');
row.true_main_1_pct = 100 * mean(tm == 1, 'omitnan');
row.true_main_3_pct = 100 * mean(tm == 3, 'omitnan');
row.pred_main_1_pct = 100 * mean(lm == 1, 'omitnan');
row.pred_main_2_pct = 100 * mean(lm == 2, 'omitnan');
```

```
1981: row.pred_main_3_pct = 100 * mean(lm == 3, 'omitnan');
1982: row.true_turn_m1_pct = 100 * mean(tt == -1, 'omitnan');
1983: row.true_turn_0_pct = 100 * mean(tt == 0, 'omitnan');
1984: row.true_turn_1_pct = 100 * mean(tt == 1, 'omitnan');
1985: row.pred_turn_m1_pct = 100 * mean(lt == -1, 'omitnan');
1986: row.pred_turn_0_pct = 100 * mean(lt == 0, 'omitnan');
1987: row.pred_turn_1_pct = 100 * mean(lt == 1, 'omitnan');
1988: row.conf_median = median(conf_main(mask), 'omitnan');
1989: row.conf_p10 = prctile(conf_main(mask), 10);
1990: row.F_min = min(F_cmd(mask));
1991: row.F_max = max(F_cmd(mask));
1992: row.omega_cmd_min = min(omega_cmd(mask));
1993: row.omega_cmd_max = max(omega_cmd(mask));
1994: end
1995:
1996: function truth = local_make_truth(theta_ground, omega_ref)
1997: truth = struct();
1998: truth.main = ones(size(theta_ground));
1999: truth.main(abs(theta_ground) > deg2rad(2.0)) = 3;
2000: truth.turn = zeros(size(omega_ref));
2001: truth.turn(omega_ref > 0.02) = 1;
2002: truth.turn(omega_ref < -0.02) = -1;
2003: end
2004:
2005: function [feature_raw, feature_norm] = local_extract_features_from_yraw(y_raw, t, params, dataset)
2006: Ts = median(diff(t));
2007: scaler = dataset.scaler;
2008: seq_skip = round(dataset.meta.skip_initial_sec / Ts);
2009: n = size(y_raw, 1);
2010: feat_dim = numel(scaler.mean);
2011: feature_raw = nan(n, feat_dim);
2012:
2013: r = local_field_or_default(params, 'wheel_radius', 0.1);
2014: W = local_field_or_default(params, 'W', 1.0);
2015: alpha_accel = Ts / (scaler.tau_accel_lp + Ts);
2016: alpha_diff = Ts / (scaler.tau_diff + Ts);
2017: lambda_pitch = exp(-Ts / scaler.tau_pitch);
2018:
2019: started = false;
2020: v_hat_prev = 0.0;
2021: dv_hat_dt_prev = 0.0;
2022: accel_x_lp_prev = 0.0;
2023: pitch_angle_est_prev = 0.0;
2024:
2025: for k = (seq_skip + 1):n
2026:     y = double(y_raw(k, :));
2027:     accel_x = y(9);
2028:     gyro_y = y(10);
2029:     gyro_z = y(11);
2030:     I_lf = y(12);
2031:     I_rr = y(13);
2032:     omega_wheel_lf = y(17);
2033:     omega_wheel_rr = y(18);
2034:     delta_lf = y(6);
2035:     delta_rr = y(7);
2036:     v_hat = r * (omega_wheel_lf + omega_wheel_rr) / 2;
2037:
2038:     if ~started
2039:         dv_hat_dt = 0.0;
2040:         accel_x_lp = accel_x;
```

```
2041:         pitch_angle_est = 0.0;
2042:         started = true;
2043:     else
2044:         dv_raw = (v_hat - v_hat_prev) / Ts;
2045:         dv_hat_dt = alpha_diff * dv_raw + (1 - alpha_diff) * dv_hat_dt_prev;
2046:         accel_x_lp = alpha_accel * accel_x + (1 - alpha_accel) * accel_x_lp_prev;
2047:         pitch_angle_est = lambda_pitch * pitch_angle_est_prev + gyro_y * Ts;
2048:     end
2049:
2050:     ws_imbalance = abs(omega_wheel_lf - omega_wheel_rr);
2051:     I_sum = abs(I_lf) + abs(I_rr);
2052:     I_diff_signed = I_lf - I_rr;
2053:     I_diff_abs = abs(I_lf) - abs(I_rr);
2054:     kappa_proxy = (tan(delta_lf) - tan(delta_rr)) / W;
2055:     accel_per_current = accel_x_lp / max(I_sum, 0.1);
2056:
2057:     feature_raw(k, :) = [accel_x, gyro_z, I_lf, I_rr, omega_wheel_lf, omega_wheel_rr, ...
2058:         delta_lf, delta_rr, gyro_y, v_hat, dv_hat_dt, ws_imbalance, ...
2059:         I_sum, I_diff_signed, I_diff_abs, accel_x_lp, kappa_proxy, ...
2060:         accel_per_current, pitch_angle_est];
2061:
2062:     v_hat_prev = v_hat;
2063:     dv_hat_dt_prev = dv_hat_dt;
2064:     accel_x_lp_prev = accel_x_lp;
2065:     pitch_angle_est_prev = pitch_angle_est;
2066: end
2067:
2068: feature_norm = (feature_raw - double(scaler.mean(:).')) ./ ...
2069:     (double(scaler.std(:).') + 1e-8);
2070: end
2071:
2072: function T = local_feature_zone_summary(zone_names, zones, t, feature_norm, feat_names)
2073: rows = repmat(struct('zone', "", 'feature', "", 'median_z', NaN, ...
2074:     'p95_abs_z', NaN, 'max_abs_z', NaN, 'pct_abs_z_gt_3', NaN), ...
2075:     numel(zone_names) * numel(feat_names), 1);
2076: idx = 0;
2077: for zi = 1:numel(zone_names)
2078:     zn = zone_names{zi};
2079:     tr = zones.(zn);
2080:     mask = t >= tr(1) & t < tr(2);
2081:     for fi = 1:numel(feat_names)
2082:         idx = idx + 1;
2083:         z = feature_norm(mask, fi);
2084:         z = z(isfinite(z));
2085:         rows(idx).zone = string(zn);
2086:         rows(idx).feature = string(feat_names{fi});
2087:         if isempty(z)
2088:             continue;
2089:         end
2090:         rows(idx).median_z = median(z);
2091:         rows(idx).p95_abs_z = prctile(abs(z), 95);
2092:         rows(idx).max_abs_z = max(abs(z));
2093:         rows(idx).pct_abs_z_gt_3 = 100 * mean(abs(z) > 3);
2094:     end
2095: end
2096: T = struct2table(rows(1:idx));
2097: end
2098:
2099: function T = local_feature_compare_to_training(dataset, feature_norm, t, zones)
2100: feat_names = dataset.feat_names(:);
```

```
X = cat(1, dataset.X_train, dataset.X_val, dataset.X_test);
y_main = [dataset.y_main_train; dataset.y_main_val; dataset.y_main_test];
y_turn = [dataset.y_turn_train; dataset.y_turn_val; dataset.y_turn_test];

cases = {
    'train_flat_straight',  y_main == 1 & y_turn == 0;
    'train_flat_left',      y_main == 1 & y_turn == 1;
    'train_slope_straight', y_main == 3 & y_turn == 0;
    'sim_pure_turn',        [];
    'sim_pure_slope',       [];
    'sim_composite',        [];
};

rows = repmat(struct('group', "", 'feature', "", 'median_z', NaN, ...
    'p95_abs_z', NaN, 'max_abs_z', NaN), numel(cases) * numel(feat_names), 1);
idx = 0;
for ci = 1:size(cases, 1)
    group = cases{ci, 1};
    train_mask = cases{ci, 2};
    if startsWith(group, 'train')
        if ~any(train_mask)
            Z = [];
        else
            Xg = X(train_mask, :, :);
            Z = reshape(Xg, [], numel(feat_names));
        end
    else
        switch group
            case 'sim_pure_turn'
                tr = zones.pure_turn;
            case 'sim_pure_slope'
                tr = zones.pure_slope;
            otherwise
                tr = zones.composite;
        end
        m = t >= tr(1) & t < tr(2);
        Z = feature_norm(m, :);
    end
    for fi = 1:numel(feat_names)
        idx = idx + 1;
        z = Z(:, fi);
        z = z(isfinite(z));
        rows(idx).group = string(group);
        rows(idx).feature = string(feat_names{fi});
        if isempty(z)
            continue;
        end
        rows(idx).median_z = median(z);
        rows(idx).p95_abs_z = prctile(abs(z), 95);
        rows(idx).max_abs_z = max(abs(z));
    end
end
T = struct2table(rows(1:idx));
end

function assessment = local_assess_closed_loop(zone_table, feature_zone)
rows = repmat(struct('check', "", 'value', NaN, 'threshold', "", ...
    'pass', false, 'comment', ""), 0, 1);

pure_turn = local_zone_or_empty(zone_table, 'pure_turn');
```

```
pure_slope = local_zone_or_empty(zone_table, 'pure_slope');
composite = local_zone_or_empty(zone_table, 'composite');
closure = local_zone_or_empty(zone_table, 'closure');
if height(closure) == 0
    closure = local_zone_or_empty(zone_table, 'closed_loop');
end

if height(pure_turn) > 0
    rows(end+1) = local_assess_row('pure_turn_false_slope_pct', ...
        pure_turn.pred_main_3_pct, '<= 5', pure_turn.pred_main_3_pct <= 5, ...
        'Flat turn should not be classified as slope.');
    rows(end+1) = local_assess_row('pure_turn_left_recall_pct', ...
        pure_turn.pred_turn_1_pct, '>= 80', pure_turn.pred_turn_1_pct >= 80, ...
        'Flat left turn should be detected before labels are used by MPC.');
    rows(end+1) = local_assess_row('pure_turn_theta_mae_deg', ...
        pure_turn.theta_mae_deg, '<= 2', pure_turn.theta_mae_deg <= 2, ...
        'Theta estimate should stay near zero on flat turns.');
end

if height(pure_slope) > 0
    rows(end+1) = local_assess_row('pure_slope_main_acc_pct', ...
        pure_slope.main_acc_pct, '>= 90', pure_slope.main_acc_pct >= 90, ...
        'Pure slope segment should be recognized consistently.');
    rows(end+1) = local_assess_row('pure_slope_theta_mae_deg', ...
        pure_slope.theta_mae_deg, '<= 2', pure_slope.theta_mae_deg <= 2, ...
        'Slope magnitude error should remain below controller-use tolerance.');
end

if height(composite) > 0
    rows(end+1) = local_assess_row('composite_turn_recall_pct', ...
        composite.pred_turn_1_pct, '>= 60', composite.pred_turn_1_pct >= 60, ...
        'Composite slope-turn segment should preserve turn information.');
end

if height(closure) > 0
    rows(end+1) = local_assess_row('closure_main_acc_pct', ...
        closure.main_acc_pct, '>= 90', closure.main_acc_pct >= 90, ...
        'Low-speed closure should not collapse into false slope labels.');
end

ood = local_feature_ood(feature_zone, closure, 'v_hat');
if isfinite(ood)
    rows(end+1) = local_assess_row('closure_v_hat_ood_pct', ...
        ood, '<= 5', ood <= 5, ...
        'High low-speed OOD rate means the dataset needs low-speed closure samples.');
end

ood = local_feature_ood(feature_zone, closure, 'accel_per_current');
if isfinite(ood)
    rows(end+1) = local_assess_row('closure_accel_per_current_ood_pct', ...
        ood, '<= 5', ood <= 5, ...
        'Large accel/current ratio OOD points to low-load sample coverage gap.');
end

assessment = struct2table(rows);
end

function row = local_assess_row(check, value, threshold, pass, comment)
row = struct('check', string(check), 'value', double(value), ...
    'threshold', string(threshold), 'pass', logical(pass), ...
```

```
2221:     'comment', string(comment));
2222: end
2223:
2224: function Z = local_zone_or_empty(T, name)
2225: Z = T(T.zone == string(name), :);
2226: end
2227:
2228: function ood = local_feature_ood(feature_zone, zone_row, feature_name)
2229: ood = NaN;
2230: if height(zone_row) == 0
2231:     return;
2232: end
2233: zone_name = zone_row.zone(1);
2234: m = feature_zone.zone == zone_name & feature_zone.feature == string(feature_name);
2235: if any(m)
2236:     ood = feature_zone.pct_abs_z_gt_3(find(m, 1, 'first'));
2237: end
2238: end
2239:
2240: function zones = local_get_zones(root, t)
2241: path_file = fullfile(root, 'data', 'paths', 'path_industrial_lite.mat');
2242: if exist(path_file, 'file') == 2
2243:     S = load(path_file, 'ref');
2244:     if isfield(S, 'ref') && isfield(S.ref, 'meta') && isfield(S.ref.meta, 'zones')
2245:         zones = S.ref.meta.zones;
2246:         return;
2247:     end
2248: end
2249: zones = struct();
2250: zones.startup = [min(t), 10];
2251: zones.golden_test = [10, 50];
2252: zones.pure_turn = [50, 78];
2253: zones.pure_slope = [78, 98];
2254: zones.composite = [98, 118];
2255: zones.closure = [118, max(t)];
2256: end
2257:
2258: function data = local_resample(logs, name, t)
2259: ts = local_timeseries(logs, name);
2260: data = squeeze(double(ts.Data));
2261: if isvector(data)
2262:     data = data(:);
2263: else
2264:     data = reshape(data, [], size(data, ndims(data)));
2265:     data = data(:);
2266: end
2267: tt = double(ts.Time(:));
2268: if numel(tt) ~= numel(data)
2269:     data = squeeze(double(ts.Data));
2270:     data = reshape(data, [], numel(tt)).';
2271: end
2272: if numel(tt) == numel(t) && max(abs(tt - t)) < 1e-12
2273:     return;
2274: end
2275: data = interp1(tt, data, t, 'linear', 'extrap');
2276: end
2277:
2278: function data = local_data(logs, name)
2279: ts = local_timeseries(logs, name);
2280: data = squeeze(double(ts.Data));
```

```
2281: end
2282:
2283: function t = local_time(logs, name)
2284: ts = local_timeseries(logs, name);
2285: t = double(ts.Time(:));
2286: end
2287:
2288: function ts = local_timeseries(logs, name)
2289: hit = [];
2290: for i = 1:logs.numElements
2291:     el = logs.getElement(i);
2292:     if strcmp(el.Name, name)
2293:         hit(end + 1) = i; %#ok<AGROW>
2294:     end
2295: end
2296: if isempty(hit)
2297:     error('ModernTCN:MissingLogSignal', 'logsout missing signal: %s', name);
2298: end
2299: % Duplicate names exist for some diag signals. Prefer the last logged signal,
2300: % which is the high-rate diagnostic line in this model.
2301: ts = logs.getElement(hit(end)).Values;
2302: end
2303:
2304: function local_write_report(report_file, result)
2305: fid = fopen(report_file, 'w');
2306: if fid < 0
2307:     warning('ModernTCN:ReportFailed', 'Cannot write report: %s', report_file);
2308:     return;
2309: end
2310: cleanup = onCleanup(@() fclose(fid));
2311:
2312: fprintf(fid, '# ModernTCN Closed-loop Diagnostic\n\n');
2313: fprintf(fid, '- out file: `%s`\n', result.out_file);
2314: fprintf(fid, '- dataset: `%s`\n\n', result.dataset_file);
2315:
2316: fprintf(fid, '## Decision Summary\n\n');
2317: fprintf(fid, '- Keep ModernTCN output diagnostic-only for now; do not feed `theta_hat` or labels
into MPC scheduling yet.\n');
2318: fprintf(fid, '- The controller is not the current bottleneck: tracking errors stay small in the
neutral-control run, while classifier errors concentrate in path zones.\n');
2319: fprintf(fid, '- Main data gap: low-load flat turns are being confused with slope, and turn labels
are missed in flat/composite turn zones.\n');
2320: fprintf(fid, '- Next training revision should add targeted short samples for flat low-load turns
and mild slope-turn composites, then rerun full-test and this closed-loop diagnostic.\n\n');
2321:
2322: fprintf(fid, '## Readiness Checks\n\n');
2323: A = result.assessment;
2324: fprintf(fid, '| check | value | threshold | pass | comment |\n');
2325: fprintf(fid, '|---|---:|---:|---:|---|\n');
2326: for i = 1:height(A)
2327:     if A.pass(i)
2328:         pass_text = "yes";
2329:     else
2330:         pass_text = "no";
2331:     end
2332:     fprintf(fid, '| %s | %.3f | %s | %s | %s |\n', ...
2333:         A.check(i), A.value(i), A.threshold(i), pass_text, A.comment(i));
2334: end
2335: fprintf(fid, '\n');
2336:
2337: fprintf(fid, '## Per-zone Metrics\n\n');
2338: fprintf(fid, '| zone | ey rmse | epsi rmse | ev rmse | theta MAE deg | main acc | turn acc | pred
main 1/2/3 | pred turn -1/0/1 |\n');
2339: fprintf(fid, '|---|---:|---:|---:|---:|---:|---:|---:|---:|\n');
2340: T = result.zone_table;
```

```
for i = 1:height(T)
    fprintf(fid, '| %s | %.4f | %.4f | %.4f | %.3f | %.1f | %.1f | %.1f/%.1f/%.1f | %.1f/%.1f/%.1f |\n', ...
        T.zone(i), T.ey_rmse(i), T.epsi_rmse(i), T.ev_rmse(i), ...
        T.theta_mae_deg(i), T.main_acc_pct(i), T.turn_acc_pct(i), ...
        T.pred_main_1_pct(i), T.pred_main_2_pct(i), T.pred_main_3_pct(i), ...
        T.pred_turn_m1_pct(i), T.pred_turn_0_pct(i), T.pred_turn_1_pct(i));
end

fprintf(fid, '\n## Top Feature Z-score by Zone\n\n');
F = result.feature_zone;
zones = unique(F.zone, 'stable');
for zi = 1:numel(zones)
    zname = zones(zi);
    Fz = F(F.zone == zname, :);
    Fz = sortrows(Fz, 'p95_abs_z', 'descend');
    fprintf(fid, '### %s\n\n', zname);
    fprintf(fid, '| feature | median z | p95 abs z | max abs z | pct abs z > 3 |\n');
    fprintf(fid, '|---|---:|---:|---:|---:|\n');
    n = min(8, height(Fz));
    for i = 1:n
        fprintf(fid, '| %s | %.3f | %.3f | %.3f | %.1f |\n', ...
            Fz.feature(i), Fz.median_z(i), Fz.p95_abs_z(i), ...
            Fz.max_abs_z(i), Fz.pct_abs_z_gt_3(i));
    end
    fprintf(fid, '\n');
end

fprintf(fid, '## Feature Distribution Compare\n\n');
groups = unique(result.feature_compare.group, 'stable');
for gi = 1:numel(groups)
    g = groups(gi);
    G = result.feature_compare(result.feature_compare.group == g, :);
    G = sortrows(G, 'p95_abs_z', 'descend');
    fprintf(fid, '### %s\n\n', g);
    fprintf(fid, '| feature | median z | p95 abs z | max abs z |\n');
    fprintf(fid, '|---|---:|---:|---:|\n');
    n = min(8, height(G));
    for i = 1:n
        fprintf(fid, '| %s | %.3f | %.3f | %.3f |\n', ...
            G.feature(i), G.median_z(i), G.p95_abs_z(i), G.max_abs_z(i));
    end
    fprintf(fid, '\n');
end
end

function y = local_rms(x)
x = x(isfinite(x));
if isempty(x)
    y = NaN;
else
    y = sqrt(mean(x.^2));
end
end

function v = local_field_or_default(s, field_name, default_value)
if isstruct(s) && isfield(s, field_name) && ~isempty(s.(field_name))
    v = s.(field_name);
else
    v = default_value;
end
```

```
2401: end
2402:
2403: function root = local_project_root()
2404: if exist('project_root', 'file') == 2
2405:     root = project_root();
2406: else
2407:     root = pwd;
2408: end
2409: end
2410:
2411: ===== FILE: src/ModernTCN/ModernTCN_replay_closed_loop_yraw.m =====
2412: function result = ModernTCN_replay_closed_loop_yraw(out_file, max_steps, cfg)
2413: %MODERNTCN_REPLAY_CLOSED_LOOP_YRAW Replay logged y_raw through a ModernTCN wrapper.
2414: %
2415: % This diagnostic does not rerun or modify the Simulink model. It reuses
2416: % diag.y_raw from an existing out.mat and calls the same
2417: % ModernTCN_State_Classifier_sim(y_raw, reset) entry used by Simulink.
2418: %
2419: % Optional cfg fields:
2420: %   seed      : default 21.
2421: %   run_tag   : results/modern_tcn/<run_tag>/modern_tcn_seed<seed>.onnx.
2422: %   onnx_file : explicit ONNX path, overrides run_tag.
2423:
2424: if nargin < 1 || isempty(out_file)
2425:     out_file = fullfile(local_project_root(), 'out.mat');
2426: end
2427: if nargin < 2 || isempty(max_steps)
2428:     max_steps = 0;
2429: end
2430: if nargin < 3 || isempty(cfg)
2431:     cfg = struct();
2432: end
2433: if exist('init_project', 'file') == 2
2434:     init_project();
2435: end
2436:
2437: root = local_project_root();
2438: cfg = local_normalize_cfg(root, cfg);
2439: params = parameters();
2440: assignin('base', 'params', params);
2441: [had_sim_cfg, old_sim_cfg] = local_install_sim_cfg(cfg);
2442: cleanup_cfg = onCleanup(@() local_restore_sim_cfg(had_sim_cfg, old_sim_cfg)); %#ok<NASGU>
2443:
2444: S = load(out_file, 'logsout');
2445: logs = S.logsout;
2446:
2447: t = local_time(logs, 'diag.y_raw');
2448: y_raw = local_data(logs, 'diag.y_raw');
2449: theta_ground = local_resample(logs, 'diag.theta_ground', t);
2450: omega_ref = local_resample(logs, 'diag.omega_ref', t);
2451:
2452: if max_steps > 0
2453:     n = min(max_steps, numel(t));
2454:     t = t(1:n);
2455:     y_raw = y_raw(1:n, :);
2456:     theta_ground = theta_ground(1:n);
2457:     omega_ref = omega_ref(1:n);
2458: else
2459:     n = numel(t);
2460: end
```

```
2461:
2462: theta_hat = nan(n, 1);
2463: label_main = nan(n, 1);
2464: label_turn = nan(n, 1);
2465: conf_main = nan(n, 1);
2466: label_main_raw = nan(n, 1);
2467: label_turn_raw = nan(n, 1);
2468: theta_hat_raw = nan(n, 1);
2469: conf_turn = nan(n, 1);
2470: main_prob = nan(n, 3);
2471: turn_prob = nan(n, 3);
2472: features = nan(n, 19);
2473: features_norm = nan(n, 19);
2474:
2475: ModernTCN_State_Classifier_sim(y_raw(1, :).', 1);
2476: fprintf('[ModernTCN replay] steps=%d | out=%s\n', n, out_file);
2477: fprintf('  onnx=%s\n', cfg.onnx_file);
2478: for k = 1:n
2479:     [theta_hat(k), label_main(k), label_turn(k), conf_main(k)] = ...
2480:         ModernTCN_State_Classifier_sim(y_raw(k, :).', 0);
2481:     debug_out = evalin('base', 'modern_tcn_out_temp');
2482:     [label_main_raw(k), label_turn_raw(k), theta_hat_raw(k), conf_turn(k), ...
2483:         main_prob(k, :), turn_prob(k, :), features(k, :), features_norm(k, :)] = ...
2484:         local_debug_fields(debug_out);
2485:     if mod(k, 1000) == 0 || k == n
2486:         fprintf('  replayed %d/%d\n', k, n);
2487:     end
2488: end
2489:
2490: truth02 = local_make_truth(theta_ground, omega_ref, 0.02);
2491: truth05 = local_make_truth(theta_ground, omega_ref, 0.05);
2492: zones = local_get_zones(root, t);
2493: zone_names = fieldnames(zones);
2494: rows = repmat(local_empty_zone_row(), numel(zone_names), 1);
2495: for i = 1:numel(zone_names)
2496:     name = zone_names{i};
2497:     tr = zones.(name);
2498:     mask = t >= tr(1) & t < tr(2);
2499:     rows(i) = local_zone_row(name, tr, mask, theta_hat, theta_ground, ...
2500:         label_main, label_turn, label_main_raw, label_turn_raw, conf_main, ...
2501:         conf_turn, turn_prob, truth02, truth05, omega_ref);
2502: end
2503: zone_table = struct2table(rows);
2504:
2505: out_dir = fullfile(root, 'results', 'modern_tcn', 'closed_loop_replay');
2506: if exist(out_dir, 'dir') ~= 7
2507:     mkdir(out_dir);
2508: end
2509: [~, base_name, ~] = fileparts(out_file);
2510: report_tag = local_report_tag(cfg);
2511: mat_file = fullfile(out_dir, sprintf('%s_modern_tcn_replay_%s.mat', base_name, report_tag));
2512: report_file = fullfile(out_dir, sprintf('%s_modern_tcn_replay_%s_report.md', base_name,
report_tag));
2513:
2514: result = struct();
2515: result.out_file = out_file;
2516: result.cfg = cfg;
2517: result.onnx_file = cfg.onnx_file;
2518: result.zone_table = zone_table;
2519: result.t = t;
2520: result.theta_hat = theta_hat;
```

```
result.theta_hat_raw = theta_hat_raw;
result.label_main = label_main;
result.label_main_raw = label_main_raw;
result.label_turn = label_turn;
result.label_turn_raw = label_turn_raw;
result.conf_main = conf_main;
result.conf_turn = conf_turn;
result.main_prob = main_prob;
result.turn_prob = turn_prob;
result.features = features;
result.features_norm = features_norm;
result.theta_ground = theta_ground;
result.omega_ref = omega_ref;
result.truth02 = truth02;
result.truth05 = truth05;
result.mat_file = mat_file;
result.report_file = report_file;

save(mat_file, 'result');
local_write_report(report_file, result);

fprintf('[ModernTCN replay] report: %s\n', report_file);
disp(zone_table(:, {'zone','theta_mae_deg','main_acc_pct','turn_acc05_pct', ...
    'left_recall05_pct','left_raw_recall05_pct','pred_main_3_pct', ...
    'pred_turn_1_pct','pred_turn_raw_1_pct','left_prob_p90'}));
end

function cfg = local_normalize_cfg(root, cfg)
default_cfg = ModernTCN_default_config(root);
if ~isfield(cfg, 'seed') || isempty(cfg.seed)
    cfg.seed = default_cfg.seed;
end
if ~isfield(cfg, 'run_tag')
    cfg.run_tag = default_cfg.run_tag;
end
if ~isfield(cfg, 'onnx_file')
    cfg.onnx_file = '';
end
if ~isfield(cfg, 'dataset_file') || isempty(cfg.dataset_file)
    cfg.dataset_file = default_cfg.dataset_file;
end
if isempty(cfg.onnx_file)
    if ~isempty(cfg.run_tag)
        cfg.onnx_file = fullfile(root, 'results', 'modern_tcn', char(cfg.run_tag), ...
            sprintf('modern_tcn_seed%d.onnx', cfg.seed));
    else
        cfg.onnx_file = default_cfg.onnx_file;
    end
end
end

function [had_cfg, old_cfg] = local_install_sim_cfg(cfg)
had_cfg = evalin('base', 'exist(''modern_tcn_sim_cfg'', ''var'')==1');
if had_cfg
    old_cfg = evalin('base', 'modern_tcn_sim_cfg');
else
    old_cfg = [];
end
assignin('base', 'modern_tcn_sim_cfg', cfg);
end
```

```
2581:
2582: function local_restore_sim_cfg(had_cfg, old_cfg)
2583: if had_cfg
2584:     assignin('base', 'modern_tcn_sim_cfg', old_cfg);
2585: else
2586:     evalin('base', 'clear modern_tcn_sim_cfg');
2587: end
2588: end
2589:
2590: function [label_main_raw, label_turn_raw, theta_hat_raw, conf_turn, main_prob, turn_prob, features,
features_norm] = ...
2591:     local_debug_fields(debug_out)
2592: label_main_raw = NaN;
2593: label_turn_raw = NaN;
2594: theta_hat_raw = NaN;
2595: conf_turn = NaN;
2596: main_prob = nan(1, 3);
2597: turn_prob = nan(1, 3);
2598: features = nan(1, 19);
2599: features_norm = nan(1, 19);
2600: if ~isstruct(debug_out)
2601:     return;
2602: end
2603: if isfield(debug_out, 'label_main_raw'); label_main_raw = double(debug_out.label_main_raw); end
2604: if isfield(debug_out, 'label_turn_raw'); label_turn_raw = double(debug_out.label_turn_raw); end
2605: if isfield(debug_out, 'theta_hat_raw'); theta_hat_raw = double(debug_out.theta_hat_raw); end
2606: if isfield(debug_out, 'conf_turn'); conf_turn = double(debug_out.conf_turn); end
2607: if isfield(debug_out, 'main_prob')
2608:     p = double(debug_out.main_prob(:)).';
2609:     main_prob(1:min(3, numel(p))) = p(1:min(3, numel(p)));
2610: end
2611: if isfield(debug_out, 'turn_prob')
2612:     p = double(debug_out.turn_prob(:)).';
2613:     turn_prob(1:min(3, numel(p))) = p(1:min(3, numel(p)));
2614: end
2615: if isfield(debug_out, 'features')
2616:     p = double(debug_out.features(:)).';
2617:     features(1:min(19, numel(p))) = p(1:min(19, numel(p)));
2618: end
2619: if isfield(debug_out, 'features_norm')
2620:     p = double(debug_out.features_norm(:)).';
2621:     features_norm(1:min(19, numel(p))) = p(1:min(19, numel(p)));
2622: end
2623: end
2624:
2625: function row = local_empty_zone_row()
2626: row = struct('zone', "", 't0', NaN, 't1', NaN, ...
2627:     'theta_mae_deg', NaN, 'theta_rmse_deg', NaN, 'theta_peak_deg', NaN, ...
2628:     'theta_hat_min_deg', NaN, 'theta_hat_max_deg', NaN, ...
2629:     'main_acc_pct', NaN, ...
2630:     'turn_acc_pct', NaN, 'turn_acc02_pct', NaN, 'turn_acc05_pct', NaN, ...
2631:     'turn_raw_acc05_pct', NaN, ...
2632:     'left_recall02_pct', NaN, 'left_recall05_pct', NaN, ...
2633:     'left_raw_recall05_pct', NaN, 'right_recall05_pct', NaN, ...
2634:     'right_raw_recall05_pct', NaN, ...
2635:     'true_main_1_pct', NaN, 'true_main_3_pct', NaN, ...
2636:     'pred_main_1_pct', NaN, 'pred_main_2_pct', NaN, 'pred_main_3_pct', NaN, ...
2637:     'true_turn_m1_pct', NaN, 'true_turn_0_pct', NaN, 'true_turn_1_pct', NaN, ...
2638:     'true_turn05_m1_pct', NaN, 'true_turn05_0_pct', NaN, 'true_turn05_1_pct', NaN, ...
2639:     'pred_turn_m1_pct', NaN, 'pred_turn_0_pct', NaN, 'pred_turn_1_pct', NaN, ...
2640:     'pred_turn_raw_m1_pct', NaN, 'pred_turn_raw_0_pct', NaN, 'pred_turn_raw_1_pct', NaN, ...
```

```
2641:     'left_prob_median', NaN, 'left_prob_p90', NaN, ...
2642:     'straight_prob_median', NaN, 'conf_median', NaN, 'conf_p10', NaN, ...
2643:     'omega_ref_min', NaN, 'omega_ref_max', NaN);
2644: end
2645:
2646: function row = local_zone_row(name, tr, mask, theta_hat, theta_ground, ...
2647:     label_main, label_turn, label_main_raw, label_turn_raw, conf_main, ...
2648:     conf_turn, turn_prob, truth02, truth05, omega_ref)
2649: row = local_empty_zone_row();
2650: row.zone = string(name);
2651: row.t0 = tr(1);
2652: row.t1 = tr(2);
2653: if nnz(mask) < 2
2654:     return;
2655: end
2656:
2657: th_err = theta_hat(mask) - theta_ground(mask);
2658: lm = round(label_main(mask));
2659: lt = round(label_turn(mask));
2660: ltr = round(label_turn_raw(mask));
2661: tm = truth05.main(mask);
2662: tt02 = truth02.turn(mask);
2663: tt05 = truth05.turn(mask);
2664: prob = turn_prob(mask, :);
2665:
2666: row.theta_mae_deg = rad2deg(mean(abs(th_err), 'omitnan'));
2667: row.theta_rmse_deg = rad2deg(local_rms(th_err));
2668: row.theta_peak_deg = rad2deg(max(abs(th_err)));
2669: row.theta_hat_min_deg = rad2deg(min(theta_hat(mask)));
2670: row.theta_hat_max_deg = rad2deg(max(theta_hat(mask)));
2671: row.main_acc_pct = 100 * mean(lm == tm, 'omitnan');
2672: row.turn_acc_pct = 100 * mean(lt == tt02, 'omitnan');
2673: row.turn_acc02_pct = row.turn_acc_pct;
2674: row.turn_acc05_pct = 100 * mean(lt == tt05, 'omitnan');
2675: row.turn_raw_acc05_pct = 100 * mean(ltr == tt05, 'omitnan');
2676: row.left_recall02_pct = local_recall_pct(lt, tt02, 1);
2677: row.left_recall05_pct = local_recall_pct(lt, tt05, 1);
2678: row.left_raw_recall05_pct = local_recall_pct(ltr, tt05, 1);
2679: row.right_recall05_pct = local_recall_pct(lt, tt05, -1);
2680: row.right_raw_recall05_pct = local_recall_pct(ltr, tt05, -1);
2681: row.true_main_1_pct = 100 * mean(tm == 1, 'omitnan');
2682: row.true_main_3_pct = 100 * mean(tm == 3, 'omitnan');
2683: row.pred_main_1_pct = 100 * mean(lm == 1, 'omitnan');
2684: row.pred_main_2_pct = 100 * mean(lm == 2, 'omitnan');
2685: row.pred_main_3_pct = 100 * mean(lm == 3, 'omitnan');
2686: row.true_turn_m1_pct = 100 * mean(tt02 == -1, 'omitnan');
2687: row.true_turn_0_pct = 100 * mean(tt02 == 0, 'omitnan');
2688: row.true_turn_1_pct = 100 * mean(tt02 == 1, 'omitnan');
2689: row.true_turn05_m1_pct = 100 * mean(tt05 == -1, 'omitnan');
2690: row.true_turn05_0_pct = 100 * mean(tt05 == 0, 'omitnan');
2691: row.true_turn05_1_pct = 100 * mean(tt05 == 1, 'omitnan');
2692: row.pred_turn_m1_pct = 100 * mean(lt == -1, 'omitnan');
2693: row.pred_turn_0_pct = 100 * mean(lt == 0, 'omitnan');
2694: row.pred_turn_1_pct = 100 * mean(lt == 1, 'omitnan');
2695: row.pred_turn_raw_m1_pct = 100 * mean(ltr == -1, 'omitnan');
2696: row.pred_turn_raw_0_pct = 100 * mean(ltr == 0, 'omitnan');
2697: row.pred_turn_raw_1_pct = 100 * mean(ltr == 1, 'omitnan');
2698: row.left_prob_median = median(prob(:, 3), 'omitnan');
2699: row.left_prob_p90 = local_prctile(prob(:, 3), 90);
2700: row.straight_prob_median = median(prob(:, 2), 'omitnan');
```

```
2701: row.conf_median = median(conf_main(mask), 'omitnan');
2702: row.conf_p10 = local_prctile(conf_main(mask), 10);
2703: row.omega_ref_min = min(omega_ref(mask));
2704: row.omega_ref_max = max(omega_ref(mask));
2705:
2706: % Keep the raw main label read alive in reports if a future diagnostic needs it.
2707: if all(~isfinite(label_main_raw(mask))) && all(~isfinite(conf_turn(mask)))
2708:     return;
2709: end
2710: end
2711:
2712: function truth = local_make_truth(theta_ground, omega_ref, omega_thresh)
2713: truth = struct();
2714: truth.main = ones(size(theta_ground));
2715: truth.main(abs(theta_ground) > deg2rad(2.0)) = 3;
2716: truth.turn = zeros(size(omega_ref));
2717: truth.turn(omega_ref > omega_thresh) = 1;
2718: truth.turn(omega_ref < -omega_thresh) = -1;
2719: truth.omega_thresh = omega_thresh;
2720: end
2721:
2722: function zones = local_get_zones(root, t)
2723: path_file = fullfile(root, 'data', 'paths', 'path_industrial_lite.mat');
2724: if exist(path_file, 'file') == 2
2725:     S = load(path_file, 'ref');
2726:     if isfield(S, 'ref') && isfield(S.ref, 'meta') && isfield(S.ref.meta, 'zones')
2727:         zones = S.ref.meta.zones;
2728:         return;
2729:     end
2730: end
2731: zones = struct();
2732: zones.startup = [min(t), 10];
2733: zones.golden_test = [10, 50];
2734: zones.pure_turn = [50, 78];
2735: zones.pure_slope = [78, 98];
2736: zones.composite = [98, 118];
2737: zones.closure = [118, max(t)];
2738: end
2739:
2740: function data = local_resample(logs, name, t)
2741: ts = local_timeseries(logs, name);
2742: data = squeeze(double(ts.Data));
2743: if isvector(data)
2744:     data = data(:);
2745: else
2746:     data = reshape(data, [], size(data, ndims(data)));
2747:     data = data(:);
2748: end
2749: tt = double(ts.Time(:));
2750: if numel(tt) ~= numel(data)
2751:     data = squeeze(double(ts.Data));
2752:     data = reshape(data, [], numel(tt)).';
2753: end
2754: if numel(tt) == numel(t) && max(abs(tt - t)) < 1e-12
2755:     return;
2756: end
2757: data = interp1(tt, data, t, 'linear', 'extrap');
2758: end
2759:
2760: function data = local_data(logs, name)
```

```
2761: ts = local_timeseries(logs, name);
2762: data = squeeze(double(ts.Data));
2763: end
2764:
2765: function t = local_time(logs, name)
2766: ts = local_timeseries(logs, name);
2767: t = double(ts.Time(:));
2768: end
2769:
2770: function ts = local_timeseries(logs, name)
2771: hit = [];
2772: for i = 1:logs.numElements
2773:     el = logs.getElement(i);
2774:     if strcmp(el.Name, name)
2775:         hit(end + 1) = i; %#ok<AGROW>
2776:     end
2777: end
2778: if isempty(hit)
2779:     error('ModernTCN:MissingLogSignal', 'logsout missing signal: %s', name);
2780: end
2781: ts = logs.getElement(hit(end)).Values;
2782: end
2783:
2784: function local_write_report(report_file, result)
2785: fid = fopen(report_file, 'w');
2786: if fid < 0
2787:     warning('ModernTCN:ReportFailed', 'Cannot write report: %s', report_file);
2788:     return;
2789: end
2790: cleanup = onCleanup(@() fclose(fid)); %#ok<NASGU>
2791:
2792: fprintf(fid, '# ModernTCN y_raw Replay Diagnostic\n\n');
2793: fprintf(fid, '- out file: `%s`\n', result.out_file);
2794: fprintf(fid, '- onnx: `%s`\n', result.onnx_file);
2795: fprintf(fid, '- turn truth thresholds: `0.02` and `0.05` rad/s; training labels use `0.05`
rad/s.\n\n');
2796: fprintf(fid, '| zone | theta MAE deg | main acc | turn acc 0.05 | left recall 0.05 | raw left
recall 0.05 | pred main 3 | pred turn 1 | raw pred turn 1 | left p90 |\n');
2797: fprintf(fid, '|---|---:|---:|---:|---:|---:|---:|---:|---:|---:|\n');
2798: T = result.zone_table;
2799: for i = 1:height(T)
2800:     fprintf(fid, '| %s | %.3f | %.1f | %.1f | %.1f | %.1f | %.1f | %.1f | %.1f | %.3f |\n', ...
2801:         T.zone(i), T.theta_mae_deg(i), T.main_acc_pct(i), T.turn_acc05_pct(i), ...
2802:         T.left_recall05_pct(i), T.left_raw_recall05_pct(i), T.pred_main_3_pct(i), ...
2803:         T.pred_turn_1_pct(i), T.pred_turn_raw_1_pct(i), T.left_prob_p90(i));
2804: end
2805: fprintf(fid, '\n## Full Zone Table\n\n');
2806: fprintf(fid, 'Saved in `%s` as `result.zone_table`.\n', result.mat_file);
2807: end
2808:
2809: function y = local_rms(x)
2810: x = x(isfinite(x));
2811: if isempty(x)
2812:     y = NaN;
2813: else
2814:     y = sqrt(mean(x.^2));
2815: end
2816: end
2817:
2818: function p = local_recall_pct(pred, truth, cls)
2819: mask = truth == cls;
2820: if ~any(mask)
```

```
2821:     p = NaN;
2822: else
2823:     p = 100 * mean(pred(mask) == cls, 'omitnan');
2824: end
2825: end
2826:
2827: function p = local_prctile(x, q)
2828: x = x(isfinite(x));
2829: if isempty(x)
2830:     p = NaN;
2831: else
2832:     p = prctile(x, q);
2833: end
2834: end
2835:
2836: function tag = local_report_tag(cfg)
2837: if isfield(cfg, 'run_tag') && ~isempty(cfg.run_tag)
2838:     tag = char(cfg.run_tag);
2839: else
2840:     tag = 'current_model';
2841: end
2842: tag = regexprep(tag, '[^A-Za-z0-9_\-]+', '_');
2843: end
2844:
2845: function root = local_project_root()
2846: if exist('project_root', 'file') == 2
2847:     root = project_root();
2848: else
2849:     root = pwd;
2850: end
2851: end
2852:
2853: ===== FILE: src/Compare/benchmark_modern_tcn_onnx_runtime.py =====
2854: """Benchmark ModernTCN single-window ONNXRuntime inference latency.
2855:
2856: The benchmark intentionally uses batch size 1 and input shape [1, 128, 19],
2857: matching the online closed-loop deployment path.  It writes raw timings and a
2858: small summary under results/compare/realtime_benchmark by default.
2859: """
2860:
2861: from __future__ import annotations
2862:
2863: import argparse
2864: import csv
2865: import json
2866: import os
2867: import platform
2868: import statistics
2869: import sys
2870: import time
2871: from pathlib import Path
2872: from typing import Dict, List
2873:
2874: import h5py
2875: import numpy as np
2876:
2877:
2878: def parse_args() -> argparse.Namespace:
2879:     root = find_project_root()
2880:     default_out = root / "results" / "compare" / "realtime_benchmark"
```

```
2881:     default_onnx = (
2882:         root
2883:         / "results"
2884:         / "modern_tcn"
2885:         / "modern_tcn_theta10_uniform_h0_v2_seed21"
2886:         / "modern_tcn_seed21.onnx"
2887:     )
2888:     default_dataset = (
2889:         root
2890:         / "data"
2891:         / "tcn"
2892:         / "ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat"
2893:     )
2894:
2895:     p = argparse.ArgumentParser(description="ModernTCN ONNXRuntime latency benchmark")
2896:     p.add_argument("--onnx-file", type=Path, default=default_onnx)
2897:     p.add_argument("--dataset-file", type=Path, default=default_dataset)
2898:     p.add_argument("--out-dir", type=Path, default=default_out)
2899:     p.add_argument("--sample-count", type=int, default=512)
2900:     p.add_argument("--warmup", type=int, default=200)
2901:     p.add_argument("--repeat", type=int, default=5000)
2902:     p.add_argument("--provider", type=str, default="CPUExecutionProvider")
2903:     p.add_argument("--intra-op-num-threads", type=int, default=1)
2904:     return p.parse_args()
2905:
2906:
2907: def find_project_root(start: Path | None = None) -> Path:
2908:     if start is None:
2909:         start = Path(__file__).resolve()
2910:     for p in (start, *start.parents):
2911:         if (p / "init_project.m").exists() and (p / "data" / "tcn").exists():
2912:             return p
2913:     cwd = Path.cwd().resolve()
2914:     for p in (cwd, *cwd.parents):
2915:         if (p / "init_project.m").exists() and (p / "data" / "tcn").exists():
2916:             return p
2917:     raise FileNotFoundError("Could not locate project root.")
2918:
2919:
2920: def read_test_windows(dataset_file: Path, sample_count: int) -> np.ndarray:
2921:     if not dataset_file.exists():
2922:         raise FileNotFoundError(f"Dataset not found: {dataset_file}")
2923:     with h5py.File(dataset_file, "r") as f:
2924:         ds = f["dataset"]["X_test"]
2925:         if ds.ndim != 3:
2926:             raise ValueError(f"Expected dataset.X_test to be 3-D, got shape={ds.shape}")
2927:         n_total = int(ds.shape[2])
2928:         n = min(max(1, int(sample_count)), n_total)
2929:         indices = np.linspace(0, n_total - 1, n, dtype=np.int64)
2930:         raw = np.asarray(ds[:, :, indices], dtype=np.float32)
2931:     # MATLAB v7.3 stores this as [feature, time, window].  ONNX expects
2932:     # [batch, time, feature].
2933:     return np.transpose(raw, (2, 1, 0)).copy()
2934:
2935:
2936: def make_session(onnx_file: Path, provider: str, intra_op_num_threads: int):
2937:     try:
2938:         import onnxruntime as ort
2939:     except ImportError as exc:
2940:         raise SystemExit("onnxruntime is not installed in the active Python.") from exc
```

```
2941:
2942:     if not onnx_file.exists():
2943:         raise FileNotFoundError(f"ONNX file not found: {onnx_file}")
2944:
2945:     opts = ort.SessionOptions()
2946:     opts.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
2947:     if intra_op_num_threads > 0:
2948:         opts.intra_op_num_threads = int(intra_op_num_threads)
2949:     session = ort.InferenceSession(str(onnx_file), sess_options=opts, providers=[provider])
2950:     return session, ort
2951:
2952:
2953: def run_benchmark(session, windows: np.ndarray, warmup: int, repeat: int) -> Dict[str, object]:
2954:     input_name = session.get_inputs()[0].name
2955:     output_names = [o.name for o in session.get_outputs()]
2956:     n = int(windows.shape[0])
2957:
2958:     for i in range(int(warmup)):
2959:         x = windows[i % n : i % n + 1]
2960:         session.run(output_names, {input_name: x})
2961:
2962:     elapsed_ms: List[float] = []
2963:     sample_idx: List[int] = []
2964:     for i in range(int(repeat)):
2965:         j = i % n
2966:         x = windows[j : j + 1]
2967:         t0 = time.perf_counter_ns()
2968:         session.run(output_names, {input_name: x})
2969:         t1 = time.perf_counter_ns()
2970:         elapsed_ms.append((t1 - t0) / 1e6)
2971:         sample_idx.append(j)
2972:
2973:     arr = np.asarray(elapsed_ms, dtype=np.float64)
2974:     return {
2975:         "input_name": input_name,
2976:         "output_names": output_names,
2977:         "sample_idx": sample_idx,
2978:         "elapsed_ms": elapsed_ms,
2979:         "summary": {
2980:             "n": int(arr.size),
2981:             "mean_ms": float(arr.mean()),
2982:             "std_ms": float(arr.std(ddof=0)),
2983:             "min_ms": float(arr.min()),
2984:             "p50_ms": float(np.percentile(arr, 50)),
2985:             "p95_ms": float(np.percentile(arr, 95)),
2986:             "p99_ms": float(np.percentile(arr, 99)),
2987:             "max_ms": float(arr.max()),
2988:         },
2989:     }
2990:
2991:
2992: def write_outputs(args: argparse.Namespace, ort, windows: np.ndarray, result: Dict[str, object]) ->
None:
2993:     out_dir = args.out_dir
2994:     out_dir.mkdir(parents=True, exist_ok=True)
2995:
2996:     raw_file = out_dir / "realtime_onnx_runtime_raw.csv"
2997:     summary_file = out_dir / "realtime_onnx_runtime_summary.csv"
2998:     metadata_file = out_dir / "realtime_onnx_runtime_metadata.json"
2999:     report_file = out_dir / "realtime_onnx_runtime_report.md"
3000:
```

```
    elapsed = result["elapsed_ms"]
    sample_idx = result["sample_idx"]
    with raw_file.open("w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["iteration", "sample_index", "elapsed_ms"])
        for i, (j, ms) in enumerate(zip(sample_idx, elapsed), start=1):
            w.writerow([i, j, f"{ms:.9g}"])

    s = result["summary"]
    row = {
        "metric": "onnxruntime_single_window",
        "n": s["n"],
        "mean_ms": s["mean_ms"],
        "std_ms": s["std_ms"],
        "min_ms": s["min_ms"],
        "p50_ms": s["p50_ms"],
        "p95_ms": s["p95_ms"],
        "p99_ms": s["p99_ms"],
        "max_ms": s["max_ms"],
        "batch_size": 1,
        "sample_count": int(windows.shape[0]),
        "warmup": int(args.warmup),
        "repeat": int(args.repeat),
        "provider": args.provider,
        "intra_op_num_threads": int(args.intra_op_num_threads),
        "onnx_file": str(args.onnx_file),
        "dataset_file": str(args.dataset_file),
    }
    with summary_file.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(row.keys()))
        w.writeheader()
        w.writerow(row)

    metadata = {
        "python": sys.version,
        "python_executable": sys.executable,
        "platform": platform.platform(),
        "processor": platform.processor(),
        "cpu_count": os.cpu_count(),
        "onnxruntime_version": ort.__version__,
        "numpy_version": np.__version__,
        "input_name": result["input_name"],
        "output_names": result["output_names"],
        "windows_shape": list(windows.shape),
    }
    metadata_file.write_text(json.dumps(metadata, indent=2), encoding="utf-8")

    with report_file.open("w", encoding="utf-8") as f:
        f.write("# ModernTCN ONNXRuntime Latency Benchmark\n\n")
        f.write(f"- onnx: `{args.onnx_file}`\n")
        f.write(f"- dataset: `{args.dataset_file}`\n")
        f.write(f"- provider: `{args.provider}`\n")
        f.write(f"- batch size: `1`\n")
        f.write(f"- warmup: `{args.warmup}`\n")
        f.write(f"- repeat: `{args.repeat}`\n\n")
        f.write("| metric | value (ms) |\n")
        f.write("|---|---:|\n")
        for key in ["mean_ms", "p50_ms", "p95_ms", "p99_ms", "max_ms"]:
            f.write(f"| {key} | {s[key]:.6g} |\n")

```

```
3061:     print(f"[onnx benchmark] summary: {summary_file}")
3062:     print(f"[onnx benchmark] report:  {report_file}")
3063:
3064:
3065: def main() -> None:
3066:     args = parse_args()
3067:     windows = read_test_windows(args.dataset_file, args.sample_count)
3068:     session, ort = make_session(args.onnx_file, args.provider, args.intra_op_num_threads)
3069:     result = run_benchmark(session, windows, args.warmup, args.repeat)
3070:     write_outputs(args, ort, windows, result)
3071:
3072:
3073: if __name__ == "__main__":
3074:     main()
3075:
3076: ===== FILE: src/Compare/run_realtime_benchmark.m =====
3077: function result = run_realtime_benchmark(cfg)
3078: %RUN_REALTIME_BENCHMARK Summarize ModernTCN closed-loop timing metrics.
3079: %
3080: % This script combines:
3081: %   1) ONNXRuntime single-window timing written by benchmark_modern_tcn_onnx_runtime.py
3082: %   2) MATLAB online replay timing for ModernTCN_state_classifier('update', ...)
3083: %   3) MPC solve-time samples already logged as diag_solve_time_ms
3084: %   4) Simulink wall-time metadata from existing closed-loop outputs
3085:
3086: if nargin < 1 || ~isstruct(cfg)
3087:     cfg = struct();
3088: end
3089:
3090: init_project();
3091: root = project_root();
3092: cfg = local_defaults(cfg, root);
3093: local_ensure_dir(cfg.out_root);
3094:
3095: params = parameters();
3096: Ts = local_field_or_default(params, 'Ts', 0.01);
3097: Ts_ms = 1000 * double(Ts);
3098:
3099: fprintf('[realtime] output root: %s\n', cfg.out_root);
3100: fprintf('[realtime] Ts = %.6g ms\n', Ts_ms);
3101:
3102: [replay_raw, replay_summary] = local_run_matlab_replay(cfg, params, Ts_ms);
3103: [mpc_raw, mpc_summary, wall_summary] = local_collect_closed_loop_timing(cfg, Ts_ms);
3104: onnx_summary = local_read_onnx_summary(cfg.onnx_summary_file, Ts_ms);
3105: summary = local_build_summary(onnx_summary, replay_summary, mpc_summary, wall_summary, Ts_ms);
3106:
3107: raw_replay_file = fullfile(cfg.out_root, 'realtime_matlab_replay_raw.csv');
3108: replay_summary_file = fullfile(cfg.out_root, 'realtime_matlab_replay_summary.csv');
3109: mpc_raw_file = fullfile(cfg.out_root, 'realtime_mpc_solve_raw.csv');
3110: mpc_summary_file = fullfile(cfg.out_root, 'realtime_mpc_solve_summary.csv');
3111: wall_summary_file = fullfile(cfg.out_root, 'realtime_simulink_wall_summary.csv');
3112: summary_file = fullfile(cfg.out_root, 'realtime_summary.csv');
3113: report_file = fullfile(cfg.out_root, 'realtime_report.md');
3114: mat_file = fullfile(cfg.out_root, 'realtime_result.mat');
3115:
3116: if ~isempty(replay_raw), writetable(replay_raw, raw_replay_file); end
3117: if ~isempty(replay_summary), writetable(replay_summary, replay_summary_file); end
3118: if ~isempty(mpc_raw), writetable(mpc_raw, mpc_raw_file); end
3119: if ~isempty(mpc_summary), writetable(mpc_summary, mpc_summary_file); end
3120: if ~isempty(wall_summary), writetable(wall_summary, wall_summary_file); end
```

```
if ~isempty(summary), writetable(summary, summary_file); end

result = struct();
result.timestamp = datestr(now, 'yyyy-mm-dd HH:MM:SS');
result.cfg = cfg;
result.Ts = Ts;
result.Ts_ms = Ts_ms;
result.onnx_summary = onnx_summary;
result.replay_raw = replay_raw;
result.replay_summary = replay_summary;
result.mpc_raw = mpc_raw;
result.mpc_summary = mpc_summary;
result.wall_summary = wall_summary;
result.summary = summary;
result.summary_file = summary_file;
result.report_file = report_file;
result.mat_file = mat_file;

save(mat_file, 'result', '-v7.3');
local_write_report(report_file, result);

fprintf('[realtime] summary: %s\n', summary_file);
fprintf('[realtime] report:  %s\n', report_file);
end

function cfg = local_defaults(cfg, root)
cfg.out_root = local_cfg(cfg, 'out_root', ...
    fullfile(root, 'results', 'compare', 'realtime_benchmark'));
cfg.warmup_sec = local_cfg(cfg, 'warmup_sec', 1.0);
cfg.predict_warmup_count = local_cfg(cfg, 'predict_warmup_count', 20);
cfg.max_replay_steps = local_cfg(cfg, 'max_replay_steps', 0);
cfg.onnx_summary_file = local_cfg(cfg, 'onnx_summary_file', ...
    fullfile(cfg.out_root, 'realtime_onnx_runtime_summary.csv'));

default_files = { ...
    fullfile(root, 'results', 'compare', 'multipath_closed_loop', ...
        'path_closed_loop_long_updown_theta10_v1', 'ModernTCN_out.mat')
    fullfile(root, 'results', 'compare', 'multipath_closed_loop', ...
        'path_closed_loop_sharp_turn_transition_theta10_v1', 'ModernTCN_out.mat')};
default_files = default_files(cellfun(@(p) exist(p, 'file') == 2, default_files));
if isempty(default_files)
    d = dir(fullfile(root, 'results', 'compare', '**', 'ModernTCN_out.mat'));
    default_files = arrayfun(@(x) fullfile(x.folder, x.name), d, 'UniformOutput', false);
end
cfg.closed_loop_files = local_cfg(cfg, 'closed_loop_files', default_files);
if ischar(cfg.closed_loop_files) || isstring(cfg.closed_loop_files)
    cfg.closed_loop_files = cellstr(cfg.closed_loop_files);
end
cfg.replay_file = local_cfg(cfg, 'replay_file', cfg.closed_loop_files{1});
end

function v = local_cfg(cfg, name, default_value)
if isstruct(cfg) && isfield(cfg, name) && ~isempty(cfg.(name))
    v = cfg.(name);
else
    v = default_value;
end
end

function local_ensure_dir(d)
```

```
if exist(d, 'dir') ~= 7
    mkdir(d);
end
end

function [raw_table, summary_table] = local_run_matlab_replay(cfg, params, Ts_ms)
raw_table = table();
summary_table = table();
if isempty(cfg.replay_file) || exist(cfg.replay_file, 'file') ~= 2
    warning('run_realtime_benchmark:MissingReplay', ...
        'Replay file not found: %s', string(cfg.replay_file));
    return;
end

S = load(cfg.replay_file, 'logsout');
yts = local_get_signal(S.logsout, 'diag.y_raw');
if isempty(yts)
    warning('run_realtime_benchmark:MissingYRaw', ...
        'diag.y_raw not found in %s', cfg.replay_file);
    return;
end

t = double(yts.Time(:));
Y = local_timeseries_rows(yts.Data, numel(t));
n = size(Y, 1);
if cfg.max_replay_steps > 0
    n = min(n, cfg.max_replay_steps);
    Y = Y(1:n, :);
    t = t(1:n);
end

clear ModernTCN_state_classifier ModernTCN_load_predictor ModernTCN_predict_window
init_timer = tic;
state = ModernTCN_state_classifier('init', params);
init_ms = 1000 * toc(init_timer);

elapsed_ms = nan(n, 1);
did_predict = false(n, 1);
ready = false(n, 1);
buffer_count = nan(n, 1);
predict_index = nan(n, 1);
predict_count = 0;

for i = 1:n
    yi = double(Y(i, :)).';
    one_timer = tic;
    [state, out] = ModernTCN_state_classifier('update', state, yi);
    elapsed_ms(i) = 1000 * toc(one_timer);
    if isstruct(out) && isfield(out, 'debug')
        dbg = out.debug;
        if isfield(dbg, 'did_predict'), did_predict(i) = logical(dbg.did_predict); end
        if isfield(dbg, 'ready'), ready(i) = logical(dbg.ready); end
        if isfield(dbg, 'buffer_count'), buffer_count(i) = double(dbg.buffer_count); end
    end
    if did_predict(i)
        predict_count = predict_count + 1;
        predict_index(i) = predict_count;
    end
end

```

```
raw_table = table( ...
    repmat(string(cfg.replay_file), n, 1), (1:n).', t(:), elapsed_ms, ...
    did_predict, ready, buffer_count, predict_index, ...
    'VariableNames', {'source_file','step_index','time_s','elapsed_ms', ...
    'did_predict','ready','buffer_count','predict_index'});

warm_mask = t(:) >= double(cfg.warmup_sec);
predict_steady_mask = warm_mask & did_predict & ...
    predict_index > double(cfg.predict_warmup_count);
rows = repmat(local_stats_row("", [], Ts_ms), 0, 1);
rows(end+1) = local_stats_row("matlab_replay_update_all", elapsed_ms(warm_mask),
Ts_ms); %#ok<AGROW>
rows(end+1) = local_stats_row("matlab_replay_update_predict_all", ...
    elapsed_ms(warm_mask & did_predict), Ts_ms); %#ok<AGROW>
rows(end+1) = local_stats_row("matlab_replay_update_predict_steady", ...
    elapsed_ms(predict_steady_mask), Ts_ms); %#ok<AGROW>
rows(end+1) = local_stats_row("matlab_replay_init_once", init_ms, Ts_ms); %#ok<AGROW>
summary_table = struct2table(rows);
summary_table.source_file = repmat(string(cfg.replay_file), height(summary_table), 1);
summary_table.warmup_sec = repmat(double(cfg.warmup_sec), height(summary_table), 1);
summary_table.predict_warmup_count = repmat(double(cfg.predict_warmup_count),
height(summary_table), 1);
end

function [raw_table, summary_table, wall_table] = local_collect_closed_loop_timing(cfg, Ts_ms)
raw_table = table();
summary_table = table();
wall_table = table();

all_values = [];
all_sources = strings(0, 1);
all_t = [];
wall_rows = repmat(local_wall_row(), 0, 1);

for i = 1:numel(cfg.closed_loop_files)
    file = cfg.closed_loop_files{i};
    if exist(file, 'file') ~= 2
        warning('run_realtime_benchmark:MissingClosedLoopFile', ...
            'Closed-loop output not found: %s', file);
        continue;
    end
    S = load(file, 'logsout', 'SimulationMetadata');
    sts = local_get_signal(S.logsout, 'diag_solve_time_ms');
    if ~isempty(sts)
        vals = double(sts.Data(:));
        t = double(sts.Time(:));
        if numel(t) ~= numel(vals)
            t = (0:numel(vals)-1).' * (Ts_ms / 1000);
        end
        mask = isfinite(vals) & t >= double(cfg.warmup_sec);
        vals = vals(mask);
        t = t(mask);
        all_values = [all_values; vals(:)]; %#ok<AGROW>
        all_sources = [all_sources; repmat(string(file), numel(vals), 1)]; %#ok<AGROW>
        all_t = [all_t; t(:)]; %#ok<AGROW>
    end

    if isfield(S, 'SimulationMetadata')
        wall_rows(end+1) = local_extract_wall_row(file, S.SimulationMetadata, S.logsout,
Ts_ms); %#ok<AGROW>
    end
end

```

```
3301: if ~isempty(all_values)
3302:     raw_table = table(all_sources, all_t, all_values, ...
3303:         'VariableNames', {'source_file','time_s','solve_time_ms'});
3304:     rows = repmat(local_stats_row("", [], Ts_ms), 0, 1);
3305:     rows(end+1) = local_stats_row("mpc_solve_time", all_values, Ts_ms); %#ok<AGROW>
3306:     summary_table = struct2table(rows);
3307:     summary_table.source_file = repmat("aggregate_closed_loop_files", height(summary_table), 1);
3308:     summary_table.warmup_sec = repmat(double(cfg.warmup_sec), height(summary_table), 1);
3309: end
3310:
3311: if ~isempty(wall_rows)
3312:     wall_table = struct2table(wall_rows);
3313: end
3314: end
3315:
3316: function T = local_read_onnx_summary(summary_file, Ts_ms)
3317: T = table();
3318: if exist(summary_file, 'file') ~= 2
3319:     warning('run_realtime_benchmark:MissingONNXSummary', ...
3320:         'ONNX summary not found: %s', summary_file);
3321:     return;
3322: end
3323: S = readtable(summary_file, 'TextType', 'string');
3324: if isempty(S)
3325:     return;
3326: end
3327: row = local_stats_row("onnxruntime_single_window", S.mean_ms(1), Ts_ms);
3328: row.n = S.n(1);
3329: row.mean_ms = S.mean_ms(1);
3330: row.std_ms = local_table_value(S, 'std_ms', 1, NaN);
3331: row.min_ms = local_table_value(S, 'min_ms', 1, NaN);
3332: row.p50_ms = S.p50_ms(1);
3333: row.p95_ms = S.p95_ms(1);
3334: row.p99_ms = local_table_value(S, 'p99_ms', 1, NaN);
3335: row.max_ms = S.max_ms(1);
3336: row.overrun_rate_vs_Ts = double(row.max_ms > Ts_ms);
3337: row.margin_p95_ms = Ts_ms - row.p95_ms;
3338: row.pass_p95 = row.p95_ms < Ts_ms;
3339: T = struct2table(row);
3340: T.source_file = string(summary_file);
3341: end
3342:
3343: function summary = local_build_summary(onnx_summary, replay_summary, mpc_summary, wall_summary,
Ts_ms)
3344: rows = repmat(local_summary_row(), 0, 1);
3345:
3346: if ~isempty(onnx_summary)
3347:     rows(end+1) = local_summary_from_stats(onnx_summary(1, :), ...
3348:         "onnxruntime_single_window", "python_onnxruntime", Ts_ms); %#ok<AGROW>
3349: end
3350: if ~isempty(replay_summary)
3351:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_predict_steady"), 1);
3352:     if ~isempty(idx)
3353:         rows(end+1) = local_summary_from_stats(replay_summary(idx, :), ...
3354:             "matlab_replay_update_predict_steady", "matlab_replay", Ts_ms); %#ok<AGROW>
3355:     end
3356:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_predict_all"), 1);
3357:     if ~isempty(idx)
3358:         rows(end+1) = local_summary_from_stats(replay_summary(idx, :), ...
3359:             "matlab_replay_update_predict_all", "matlab_replay", Ts_ms); %#ok<AGROW>
3360:     end
```

```
3361:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_all"), 1);
3362:     if ~isempty(idx)
3363:         rows(end+1) = local_summary_from_stats(replay_summary(idx, :), ...
3364:             "matlab_replay_update_all", "matlab_replay", Ts_ms); %#ok<AGROW>
3365:     end
3366: end
3367: if ~isempty(mpc_summary)
3368:     rows(end+1) = local_summary_from_stats(mpc_summary(1, :), ...
3369:         "mpc_solve_time", "closed_loop_logsout", Ts_ms); %#ok<AGROW>
3370: end
3371:
3372: if ~isempty(onnx_summary) && ~isempty(mpc_summary)
3373:     rows(end+1) = local_cycle_row("cycle_onnxruntime_plus_mpc", ...
3374:         onnx_summary(1, :), mpc_summary(1, :), "p95 sum of ONNXRuntime and MPC",
Ts_ms); %#ok<AGROW>
3375: end
3376: if ~isempty(replay_summary) && ~isempty(mpc_summary)
3377:     idx = find(strcmp(string(replay_summary.metric), "matlab_replay_update_predict_steady"), 1);
3378:     if ~isempty(idx)
3379:         rows(end+1) = local_cycle_row("cycle_matlab_replay_plus_mpc", ...
3380:             replay_summary(idx, :), mpc_summary(1, :), "p95 sum of MATLAB replay and MPC",
Ts_ms); %#ok<AGROW>
3381:     end
3382: end
3383:
3384: if ~isempty(wall_summary)
3385:     row = local_summary_row();
3386:     row.metric = "simulink_wall_per_step";
3387:     row.source = "SimulationMetadata.TimingInfo";
3388:     row.n = height(wall_summary);
3389:     vals = wall_summary.wall_ms_per_step;
3390:     row.mean_ms = mean(vals, 'omitnan');
3391:     row.p50_ms = median(vals, 'omitnan');
3392:     row.p95_ms = max(vals);
3393:     row.max_ms = max(vals);
3394:     row.margin_p95_ms = Ts_ms - row.p95_ms;
3395:     row.pass_p95 = row.p95_ms < Ts_ms;
3396:     row.note = "desktop simulation wall time, not embedded controller compute time";
3397:     rows(end+1) = row; %#ok<AGROW>
3398: end
3399:
3400: summary = struct2table(rows);
3401: end
3402:
3403: function row = local_stats_row(metric, values, Ts_ms)
3404: values = double(values(:));
3405: values = values(isfinite(values));
3406: row = struct();
3407: row.metric = string(metric);
3408: row.n = numel(values);
3409: row.mean_ms = NaN;
3410: row.std_ms = NaN;
3411: row.min_ms = NaN;
3412: row.p50_ms = NaN;
3413: row.p95_ms = NaN;
3414: row.p99_ms = NaN;
3415: row.max_ms = NaN;
3416: row.overrun_rate_vs_Ts = NaN;
3417: row.margin_p95_ms = NaN;
3418: row.pass_p95 = false;
3419: if isempty(values)
3420:     return;
```

```
3421: end
3422: row.mean_ms = mean(values);
3423: row.std_ms = std(values, 0);
3424: row.min_ms = min(values);
3425: row.p50_ms = prctile(values, 50);
3426: row.p95_ms = prctile(values, 95);
3427: row.p99_ms = prctile(values, 99);
3428: row.max_ms = max(values);
3429: row.overrun_rate_vs_Ts = mean(values > Ts_ms);
3430: row.margin_p95_ms = Ts_ms - row.p95_ms;
3431: row.pass_p95 = row.p95_ms < Ts_ms;
3432: end
3433:
3434: function row = local_summary_row()
3435: row = struct();
3436: row.metric = "";
3437: row.source = "";
3438: row.n = NaN;
3439: row.mean_ms = NaN;
3440: row.p50_ms = NaN;
3441: row.p95_ms = NaN;
3442: row.max_ms = NaN;
3443: row.overrun_rate_vs_Ts = NaN;
3444: row.margin_p95_ms = NaN;
3445: row.pass_p95 = false;
3446: row.note = "";
3447: end
3448:
3449: function row = local_summary_from_stats(T, metric, source, Ts_ms)
3450: row = local_summary_row();
3451: row.metric = string(metric);
3452: row.source = string(source);
3453: row.n = T.n(1);
3454: row.mean_ms = T.mean_ms(1);
3455: row.p50_ms = T.p50_ms(1);
3456: row.p95_ms = T.p95_ms(1);
3457: row.max_ms = T.max_ms(1);
3458: row.overrun_rate_vs_Ts = T.overrun_rate_vs_Ts(1);
3459: row.margin_p95_ms = Ts_ms - row.p95_ms;
3460: row.pass_p95 = row.p95_ms < Ts_ms;
3461: end
3462:
3463: function row = local_cycle_row(metric, A, B, note, Ts_ms)
3464: row = local_summary_row();
3465: row.metric = string(metric);
3466: row.source = "computed_sum";
3467: row.n = min(A.n(1), B.n(1));
3468: row.mean_ms = A.mean_ms(1) + B.mean_ms(1);
3469: row.p50_ms = A.p50_ms(1) + B.p50_ms(1);
3470: row.p95_ms = A.p95_ms(1) + B.p95_ms(1);
3471: row.max_ms = A.max_ms(1) + B.max_ms(1);
3472: row.overrun_rate_vs_Ts = NaN;
3473: row.margin_p95_ms = Ts_ms - row.p95_ms;
3474: row.pass_p95 = row.p95_ms < Ts_ms;
3475: row.note = string(note);
3476: end
3477:
3478: function row = local_wall_row()
3479: row = struct();
3480: row.source_file = "";
```

```
row.num_steps = NaN;
row.execution_wall_s = NaN;
row.total_wall_s = NaN;
row.wall_ms_per_step = NaN;
row.execution_to_model_time_ratio = NaN;
end

function row = local_extract_wall_row(file, meta, logsout, Ts_ms)
row = local_wall_row();
row.source_file = string(file);
row.num_steps = local_num_steps(logsout);
if isprop(meta, 'TimingInfo') || isfield(meta, 'TimingInfo')
    ti = meta.TimingInfo;
    row.execution_wall_s = local_field_or_default(ti, 'ExecutionElapsedWallTime', NaN);
    row.total_wall_s = local_field_or_default(ti, 'TotalElapsedWallTime', NaN);
end
if isfinite(row.execution_wall_s) && row.num_steps > 0
    row.wall_ms_per_step = 1000 * row.execution_wall_s / row.num_steps;
    model_time_s = row.num_steps * Ts_ms / 1000;
    row.execution_to_model_time_ratio = row.execution_wall_s / model_time_s;
end
end

function n = local_num_steps(logsout)
n = NaN;
for i = 1:logsout.numElements
    e = logsout.get(i);
    if isa(e.Values, 'timeseries')
        n = numel(e.Values.Time);
        return;
    end
end
end

function ts = local_get_signal(logsout, name)
ts = [];
if ~isa(logsout, 'Simulink.SimulationData.Dataset')
    return;
end
for i = 1:logsout.numElements
    e = logsout.get(i);
    if strcmp(e.Name, name)
        ts = e.Values;
        return;
    end
end
end

function Y = local_timeseries_rows(data, n_time)
D = squeeze(data);
sz = size(D);
if isvector(D)
    Y = double(D(:));
    return;
end
if sz(1) == n_time
    Y = double(reshape(D, n_time, []));
elseif sz(end) == n_time
    Y = double(reshape(D, [], n_time).');
elseif numel(sz) >= 2 && sz(2) == n_time
```

```
3541:     Y = double(reshape(permute(D, [2 1 3:ndims(D)]), n_time, []));
3542: else
3543:     error('run_realtime_benchmark:BadTimeseriesShape', ...
3544:         'Cannot align data shape %s with %d time samples.', mat2str(sz), n_time);
3545: end
3546: end
3547:
3548: function v = local_table_value(T, name, row, default_value)
3549: if ismember(name, T.Properties.VariableNames)
3550:     v = T.(name)(row);
3551: else
3552:     v = default_value;
3553: end
3554: end
3555:
3556: function v = local_field_or_default(s, name, default_value)
3557: if isstruct(s) && isfield(s, name)
3558:     v = s.(name);
3559: else
3560:     try
3561:         v = s.(name);
3562:     catch
3563:         v = default_value;
3564:     end
3565: end
3566: end
3567:
3568: function local_write_report(report_file, result)
3569: fid = fopen(report_file, 'w');
3570: if fid < 0
3571:     warning('run_realtime_benchmark:ReportFailed', 'Cannot write %s', report_file);
3572:     return;
3573: end
3574: cleanup = onCleanup(@() fclose(fid));
3575:
3576: fprintf(fid, '# Real-Time Benchmark\n\n');
3577: fprintf(fid, '- timestamp: `%s`\n', result.timestamp);
3578: fprintf(fid, '- Ts: `%.6g ms`\n', result.Ts_ms);
3579: fprintf(fid, '- output root: `%s`\n\n', result.cfg.out_root);
3580:
3581: fprintf(fid, '## Summary\n\n');
3582: local_write_markdown_table(fid, result.summary);
3583:
3584: fprintf(fid, '\n## Notes\n\n');
3585: fprintf(fid, '- `onnxruntime_single_window` is pure batch=1 ONNXRuntime inference.\n');
3586: fprintf(fid, '- `matlab_replay_update_predict_steady` replays closed-loop `diag.y_raw` through the
MATLAB online wrapper after the sliding window is ready and after the configured first-predict warmup
count.\n');
3587: fprintf(fid, '- `mpc_solve_time` comes from `diag_solve_time_ms` in closed-loop logs after the
configured warmup period.\n');
3588: fprintf(fid, '- `simulink_wall_per_step` is desktop Simulink wall time and is not used as embedded
controller compute time.\n');
3589: end
3590:
3591: function local_write_markdown_table(fid, T)
3592: if isempty(T)
3593:     fprintf(fid, '_No timing summary available._\n');
3594:     return;
3595: end
3596: fprintf(fid, '| metric | mean ms | p50 ms | p95 ms | max ms | p95 margin ms | pass p95 |\n');
3597: fprintf(fid, '|---|---:|---:|---:|---:|---:|---:|\n');
3598: for i = 1:height(T)
3599:     fprintf(fid, '| %s | %.6g | %.6g | %.6g | %.6g | %.6g | %d |\n', ...
3600:         string(T.metric(i)), T.mean_ms(i), T.p50_ms(i), T.p95_ms(i), ...
```

```
3601:         T.max_ms(i), T.margin_p95_ms(i), T.pass_p95(i));
3602: end
3603: end
```